

Methodologies, level 2 — intuition and usage

Felipe Santana

Contents

Welch's t-test	2
Oaxaca-Blinder decomposition	5
RIF (Recentered Influence Function) decomposition	9
Inequality decomposition (Theil, Gini, weighted quantiles)	13
Duncan Dissimilarity Index	18
Structural Break Test (Chow-Type)	21
Difference-in-Differences (DiD)	24
Rotating Panel Transition Analysis	27
A/B testing fundamentals	30
Sample Ratio Mismatch (SRM) test	34
Covariate balance and covariate adjustment	38
Non-inferiority testing	42
Heterogeneity testing and novelty-effect decay	46
Statistical power and minimum detectable effect (MDE)	49
Spatial spillover and cannibalization analysis	53
Cluster-robust standard errors	57
Randomization inference (permutation test)	60
Wild-cluster bootstrap	63
Cohort and retention decomposition	66
Growth decomposition (total, comparable, and price-volume-mix)	70

Poisson regression with fixed effects	74
Weighted least squares (WLS) trend with confidence-interval classification	78
Logistic regression	81

Welch's t-test

1. What problem does this solve?

You have two groups and notice the average of some variable differs between them — say, the average wait time in one queue is longer than in another. The question that matters is: is that difference real, in the sense of reflecting something happening in the population, or could it have arisen just from the luck of who ended up in each sample? More specifically: can you answer that reliably when the two groups don't have the same sample size or the same internal variability?

2. Intuition

Even if two populations had exactly the same mean, random samples drawn from them would almost never produce identical sample means — there is always sampling noise. Welch's t-test measures how unusual the observed difference between the means is, accounting for each sample's size and its internal spread, without requiring that spread to be equal across the two groups.

That last part is what separates Welch from the classic Student's t-test: the Student version assumes both populations share the same variance — a mathematically convenient assumption, but often unrealistic. Different groups tend to have different variability, especially when they also differ in sample size.

3. Plain-language explanation

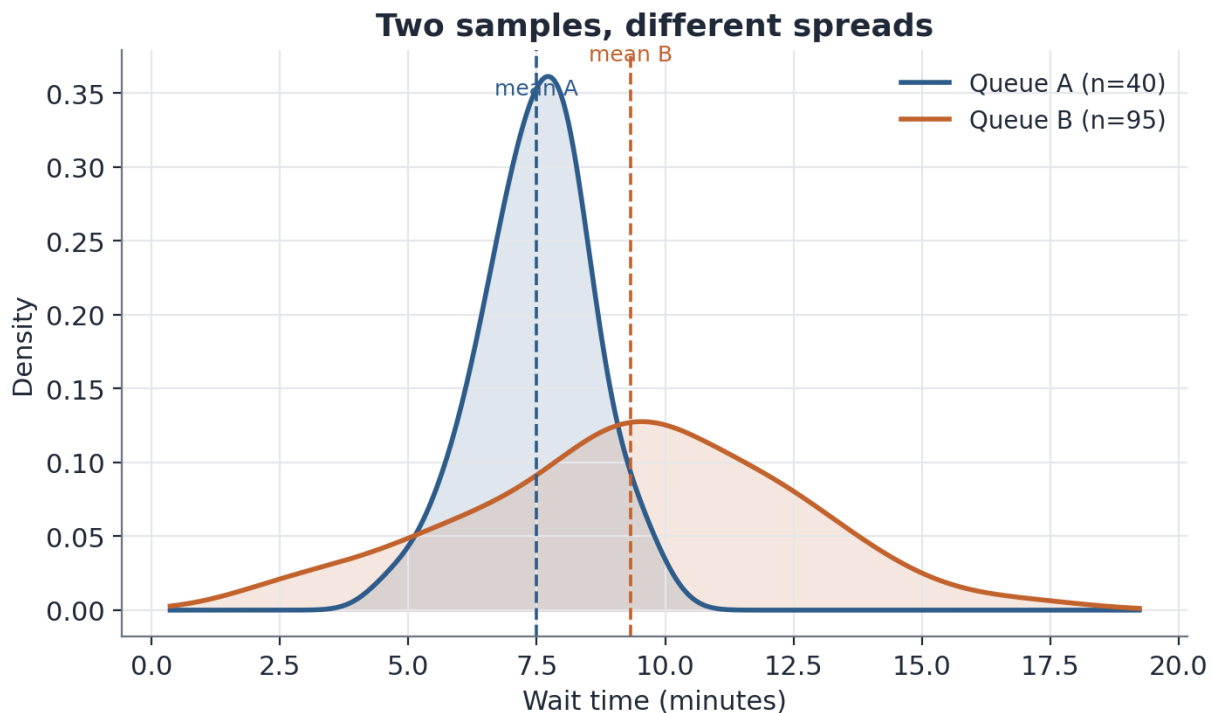
The test computes a statistic that summarizes “how many standard errors away from zero” the observed difference sits. The larger this distance (in absolute value), the more unusual it would be to see this difference if the population means were actually equal. That statistic is compared against a reference distribution (a t-distribution, with degrees of freedom computed in a specific way) to produce a p-value.

4. Easy conceptual example

Picture two checkout lines at an event. Line A has few attendants, but wait times are fairly predictable. Line B has more attendants, but wait times vary a lot — sometimes fast, sometimes stuck for a while. Even with very different sample sizes and variability between the two lines, Welch's test can still compare their averages reliably.

5. How it works, broadly

1. Compute the mean and standard deviation of each group separately.
2. Compute the standard error of the difference between means, combining the two variances without assuming they're equal.
3. Divide the observed difference in means by that standard error to get the t statistic.



Illustrative data, not drawn from any real project.

Figure 1: Two sample distributions with different spreads, showing each group's mean

4. The degrees of freedom don't come from a simple formula the way they do for Student's test — they're computed in a way that weights how much uncertainty each group contributes (the Welch-Satterthwaite approximation), typically landing on a non-integer number.
5. Compare the t statistic to the reference distribution to get the p-value.

6. What the result means

A low p-value (typically below 0.05) indicates that a difference this size would be rare if the population means were actually equal — that's evidence in favor of a real difference. A high p-value doesn't prove the means are equal; it just means the data don't carry strong evidence of a difference.

7. How to interpret it

The p-value measures evidence against the hypothesis of equal means, not the size or practical importance of the difference. A small gap can be statistically significant with a large sample, and a large gap can fail to be significant with a small or highly variable sample. Always look at the effect size alongside the p-value — never just one or the other.

8. When it's useful

When you need to compare averages between two groups that likely have different variability — which is the rule, not the exception, in real-world data. A common case is tracking, across many time periods, whether the average income gap between two population groups stays statistically

significant period after period, without assuming upfront that both groups have the same income spread.

9. Important caveats

- The test assumes reasonably independent observations; data with cluster structure (several observations from the same household or region, say) needs a separate adjustment to the standard error.
- With very small samples and heavily skewed distributions, the approximation the test relies on becomes less reliable.
- Statistical significance isn't the same as practical relevance — always report the size of the difference, not just the p-value.

10. A small worked example

Suppose two service queues, with wait times in minutes:

- Queue A: $n = 40$, mean = 8.1 min, standard deviation = 1.3 min.
- Queue B: $n = 95$, mean = 9.6 min, standard deviation = 3.4 min.

The observed difference in means is 1.5 minutes. Because Queue B has a much larger spread, Welch's test weights that extra uncertainty when computing the standard error of the difference, producing a t statistic of about 3.1, with effective degrees of freedom around 61 (a non-integer number, the result of weighting the two samples). The corresponding p-value is well below 0.01 — strong evidence that Queue B really does have a longer average wait, not just sampling luck.

11. Simple code example

```
import numpy as np
from scipy import stats

queue_a = np.array([7.9, 8.4, 7.5, 9.0, 8.2, 7.8, 8.6, 7.7, 8.3, 8.0]) # illustrative
queue_b = np.array([9.1, 12.4, 6.8, 10.2, 9.9, 7.3, 15.0, 8.5, 9.7, 10.8]) # illustrative

result = stats.ttest_ind(queue_a, queue_b, equal_var=False) # equal_var=False = Welch
print(f"t = {result.statistic:.2f}, p-value = {result.pvalue:.4f}")
```

The `equal_var=False` argument is what tells SciPy to use Welch instead of the classic Student's test — a one-line difference that avoids an assumption which, in practice, rarely holds.

Oaxaca-Blinder decomposition

1. What problem does this solve?

Two groups earn different average wages. Part of that difference might come from the groups having, on average, different characteristics — more or fewer years of education, say. But does that explain the whole difference, or is something left over even after putting those characteristics on the same footing?

2. Intuition

Imagine you want to compare the average wage of two groups, but you know they also differ, on average, in education. Simply comparing average wages mixes two things together: the education difference itself, and any difference in how each group is paid per unit of education. Oaxaca-Blinder decomposition separates these two sources, answering the question: “if Group B had Group A’s average education, but kept being paid according to its own wage structure, what average wage would we expect for Group B?”

3. Plain-language explanation

The method fits a statistical model (typically a linear regression) relating wages to observable characteristics, separately for each group. It then uses these two models to simulate a counterfactual “what if” scenario, splitting the total difference in means into two parts:

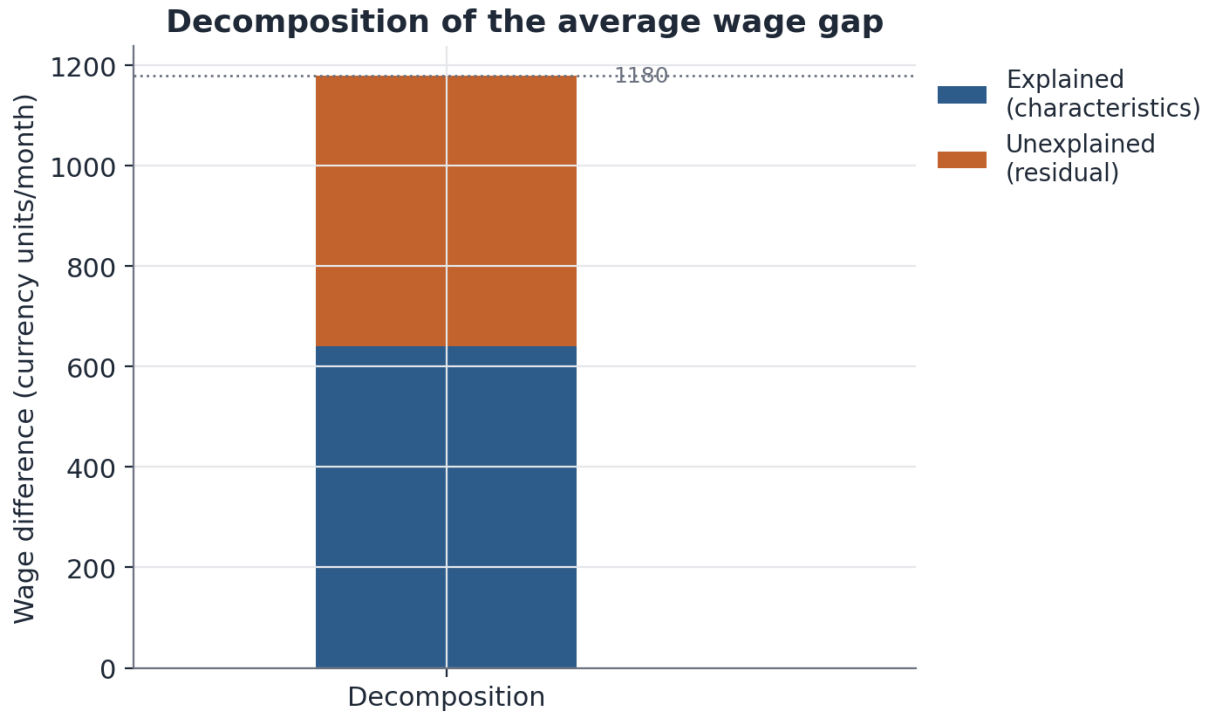
- **Explained component:** the part of the difference that comes from the groups having, on average, different characteristics.
- **Unexplained component:** what’s left over — differences in how each characteristic translates into wages for each group, plus anything not captured by the measured characteristics.

4. Easy conceptual example

Picture two sales teams at a company. Team A has, on average, more years of experience than Team B. If Team A also earns more on average, part of that difference might simply reflect the experience gap — more experience, more pay, equally in both teams. The decomposition separates how much of the gap comes from that experience difference, and how much is left over even when comparing people with the same experience across both teams.

5. How it works, broadly

1. Fit a separate regression model for each group, relating the outcome of interest (wage, say) to observable characteristics (education, age, occupation, etc.).
2. Compute the total difference in means between the groups.
3. Using the coefficients from one of the models (or a weighted combination of both, depending on the variant of the method), build the counterfactual scenario: what would Group B’s average wage be if it had Group A’s average characteristics, but kept its own wage structure.
4. The difference between Group A’s observed wage and this counterfactual is the explained component; the difference between the counterfactual and Group B’s observed wage is the unexplained component.



Illustrative data, not drawn from any real project.

Figure 2: Stacked bar showing the total wage gap split into explained and unexplained components

6. What the result means

The explained component shows how much of the difference in means can be attributed to differences in average measured characteristics. The unexplained component shows what’s left over after putting those characteristics on the same footing — it’s often read as a possible signal of discrimination or other unobserved factors, but that reading requires much stronger assumptions than the method alone guarantees: there may be wage-relevant characteristics that simply weren’t measured, and including them would shift how the total splits between the two parts.

7. How to interpret it

The method decomposes an observed difference — it doesn’t prove causation or isolate discrimination. Saying “the unexplained component is discrimination” assumes every wage-relevant characteristic was measured and included in the model, which is rarely true in practice (omitted-variable bias). The unexplained component is better described as “what couldn’t be attributed to the observed characteristics” — an important difference between failing to explain something and being certain of its cause.

8. When it’s useful

When you want to understand the composition of a gap between groups — for instance, comparing the average wage difference between demographic groups, separating how much comes from compositional differences (education, age, occupation type) and how much remains even after controlling for those variables.

9. Important caveats

- **Omitted-variable bias:** if a relevant characteristic wasn't measured, it ends up hidden inside the unexplained component.
- **Choice of reference group:** in some variants of the method, the result can shift depending on which group is used as the reference for building the counterfactual — it's worth checking the specification used.
- **The decomposition only looks at the mean:** if the gap is very different at different points of the distribution (larger at the top of the income scale, say), a mean decomposition won't capture that — for that question, methods like RIF decomposition are more appropriate.
- **Causality:** the method is descriptive/accounting, not causal — it doesn't determine whether the differences in characteristics themselves have a causal origin.

10. A small worked example

Consider a hypothetical scenario: two teams at a company, with average monthly wage (in fictional currency units) and average years of experience:

- Team A: average wage = 5,800, average experience = 8 years.
- Team B: average wage = 4,620, average experience = 5 years.

The total gap is 1,180. Suppose the fitted model shows that, holding Team A's wage structure, each additional year of experience corresponds to an average increase of about 210 units. If Team B had Team A's average experience (8 years instead of 5), its counterfactual wage would be around $4,620 + 3 \times 210 = 5,250$. That splits the total gap into:

- **Explained component** (experience difference): $5,250 - 4,620 = 630$.
- **Unexplained component:** $5,800 - 5,250 = 550$.

In other words, roughly half the gap comes from the average experience difference between the teams, and the other half remains even when comparing people with the same experience.

11. Simple code example

```
import pandas as pd
import statsmodels.formula.api as smf

# illustrative person-level data
df_a = pd.DataFrame({"wage": [...], "experience": [...]} ) # Team A
df_b = pd.DataFrame({"wage": [...], "experience": [...]} ) # Team B

model_a = smf.ols("wage ~ experience", data=df_a).fit()
model_b = smf.ols("wage ~ experience", data=df_b).fit()

mean_exp_a = df_a["experience"].mean()
mean_exp_b = df_b["experience"].mean()

# counterfactual: Team B with Team A's average experience,
# paid according to Team B's own wage structure
counterfactual_wage = model_b.params["Intercept"] + model_b.params["experience"] * mean_exp_a
```

```
explained = counterfactual_wage - df_b["wage"].mean()
unexplained = df_a["wage"].mean() - counterfactual_wage
```

Dedicated packages (like the `oaxaca` package in R, or Python implementations built on top of `statsmodels`) automate this calculation and also provide standard errors for each component of the decomposition.

RIF (Recentered Influence Function) decomposition

1. What problem does this solve?

Oaxaca-Blinder decomposition splits the difference in means between two groups into an explained and an unexplained part — but what if the gap between the groups isn’t the same everywhere in the distribution? What if the gap is small among low earners but large among high earners (or the other way around)? RIF decomposition answers that question by extending the Oaxaca-Blinder logic beyond the mean.

2. Intuition

The mean is just one of several possible summaries of a distribution. Other summaries — the median, the 10th percentile, the 90th percentile — can also be compared between groups and decomposed into “how much comes from differences in characteristics” and “how much is left over.” The technical challenge is that these other summaries (called distributional statistics) aren’t simple averages of observed variables, so the regression machinery used in Oaxaca-Blinder doesn’t apply to them directly. The recentered influence function is what solves that problem: it transforms each observation into a measure of “how much it influences” that specific summary of the distribution, in a way such that the average of this transformed measure recovers the summary itself — and once transformed, the Oaxaca-Blinder logic works again.

3. Plain-language explanation

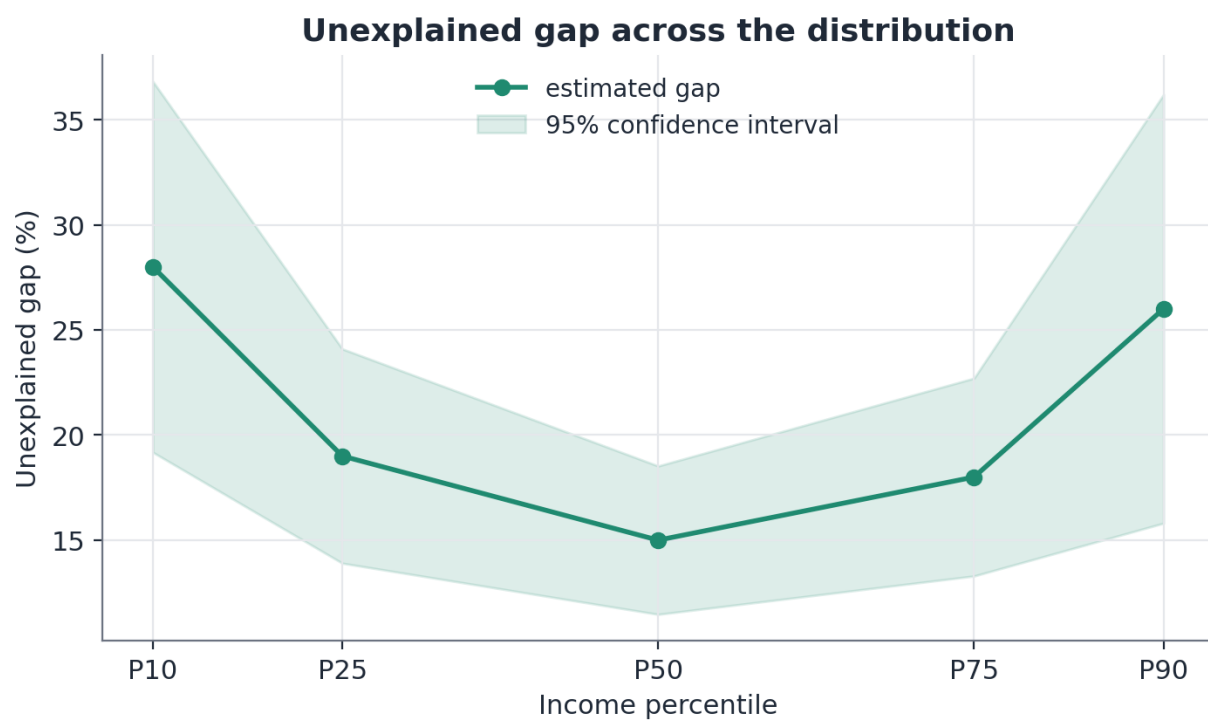
An influence function measures, for a statistic of interest (the median, say), how much that statistic would change if one specific observation were slightly perturbed. The “recentered” version is adjusted so that the average of the influence function, computed over the whole sample, equals exactly the original value of the statistic. That’s what lets you treat this transformed measure as if it were an ordinary variable and apply regression and Oaxaca-Blinder decomposition on top of it — except now you’re decomposing the difference in the statistic of interest (median, 90th percentile, etc.) instead of the difference in means of the original variable.

4. Easy conceptual example

Imagine the wage gap between two groups, looking only at the mean, is 15%. RIF decomposition might reveal that at the 10th percentile (among lower earners), the gap is 28%, while at the 50th percentile (the median) it drops to 15%, and climbs back up to 26% at the 90th percentile (among higher earners). That pattern — a larger gap at the tails of the distribution than in the middle — is common enough to have its own names in the literature: “sticky floor” (when the gap is larger at the bottom) and “glass ceiling” (when the gap is larger at the top).

5. How it works, broadly

1. Choose the distributional statistic of interest (median, a specific percentile, the Gini index, etc.).
2. Compute the recentered influence function of that statistic for every observation in the sample.
3. Use that transformed measure as the outcome variable in a regression, separately for each group — exactly as in the first step of Oaxaca-Blinder, but with the transformed variable in place of the original one.



Illustrative data, not drawn from any real project.

Figure 3: Estimated unexplained gap at different percentiles of the distribution, showing the gap is larger at the tails

4. Apply the same counterfactual decomposition logic from Oaxaca-Blinder on top of that regression, obtaining explained and unexplained components for that specific point of the distribution.
5. Repeat across several points of the distribution (several percentiles, say) to build a full profile of the gap across the distribution.

6. What the result means

For each chosen point of the distribution, the result shows how much of the gap at that point comes from differences in characteristics and how much is left over — just like Oaxaca-Blinder, but point by point instead of only at the mean. Looking at several points together shows whether the gap is uniform across the distribution or concentrated in certain regions.

7. How to interpret it

Just as in Oaxaca-Blinder, the unexplained component at each point shouldn't automatically be read as discrimination — the same caveat about omitted variables applies, point by point. Precision of the estimate also tends to vary across the distribution: points at the tails (very low or very high percentiles) typically have fewer nearby observations and wider margins of error — results there deserve a more cautious reading.

8. When it's useful

When the question isn't just “is there an average gap?” but “is that gap the same at every level of the distribution, or is it concentrated in certain ranges?” A typical use: applying this technique to show that a residual gap between demographic groups in an income distribution is larger both among low earners and among high earners, and comparatively smaller in the middle of the distribution.

9. Important caveats

- Larger standard errors at the tails of the distribution demand extra caution when interpreting differences there.
- The choice of which points of the distribution to report (just the median? a full grid of percentiles?) shapes the narrative — it's worth reporting a representative set, not just the most “convenient” point for a given story.
- Just as with Oaxaca-Blinder, the decomposition is an accounting exercise, not a causal one.

10. A small worked example

Suppose, in a hypothetical scenario, the unexplained (residual) wage gap between two groups, estimated at five percentiles of the distribution:

Percentile	Unexplained gap
P10	28%
P25	19%
P50	15%
P75	18%
P90	26%

The U-shaped pattern — larger at the tails, smaller in the middle — suggests that even comparing people with similar characteristics, the gap isn't constant across the distribution: it's more pronounced both at the bottom and at the top of the wage distribution than in the middle range.

11. Simple code example

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from scipy.stats import gaussian_kde

def rif_quantile(y, q):
    """Recentered influence function for quantile q (0 to 1)."""
    quantile_value = np.quantile(y, q)
    density = gaussian_kde(y)(quantile_value)[0]
    indicator = (y <= quantile_value).astype(float)
    return quantile_value + (q - indicator) / density

df_a = pd.DataFrame({"wage": [...], "education": [...]} ) # Group A
df_b = pd.DataFrame({"wage": [...], "education": [...]} ) # Group B

df_a["rif_median"] = rif_quantile(df_a["wage"].values, 0.5)
df_b["rif_median"] = rif_quantile(df_b["wage"].values, 0.5)

model_a = smf.ols("rif_median ~ education", data=df_a).fit()
model_b = smf.ols("rif_median ~ education", data=df_b).fit()
# from here, the decomposition follows the same steps as Oaxaca-Blinder
```

In practice, dedicated packages (like `rifreg` in R/Stata) handle the details of density estimation and standard-error calculation, which need more care than the simplified version above.

Inequality decomposition (Theil, Gini, weighted quantiles)

1. What problem does this solve?

When people talk about “income inequality,” it’s easy to think of it as a single number — but where does it come from? How much of it is because different groups (by region, by gender, by any other split) have different average incomes, and how much is because, within each group, there’s already a lot of variation between low and high earners? This family of tools — the Theil index, the Gini index, and weighted quantiles — helps answer those questions.

2. Intuition

Think of two sources of inequality that can coexist. First: different groups can have different average incomes from one another — that’s “between-group” inequality. Second: even within a single group, some people earn much more than others — that’s “within-group” inequality. The Theil index has a valuable mathematical property: it can split these two components exactly, with no leftover and no double-counting. The Gini index, on the other hand, measures income concentration within a single distribution (a group, or the whole population), on a scale from 0 (perfect equality) to 1 (one individual has everything, everyone else has nothing). Weighted quantiles are simply cutoff points — “what defines the richest 10% of this group?” — that use each observation’s sampling weight to correctly estimate those cutoffs in a survey with a complex sampling design.

3. Plain-language explanation

The Theil index computes, essentially, an entropy-based dispersion measure — the more unequal the distribution, the higher the index. Its property of exact decomposability lets you write:

$$\text{Total inequality} = \text{Between-group inequality} + \text{Within-group inequality}$$

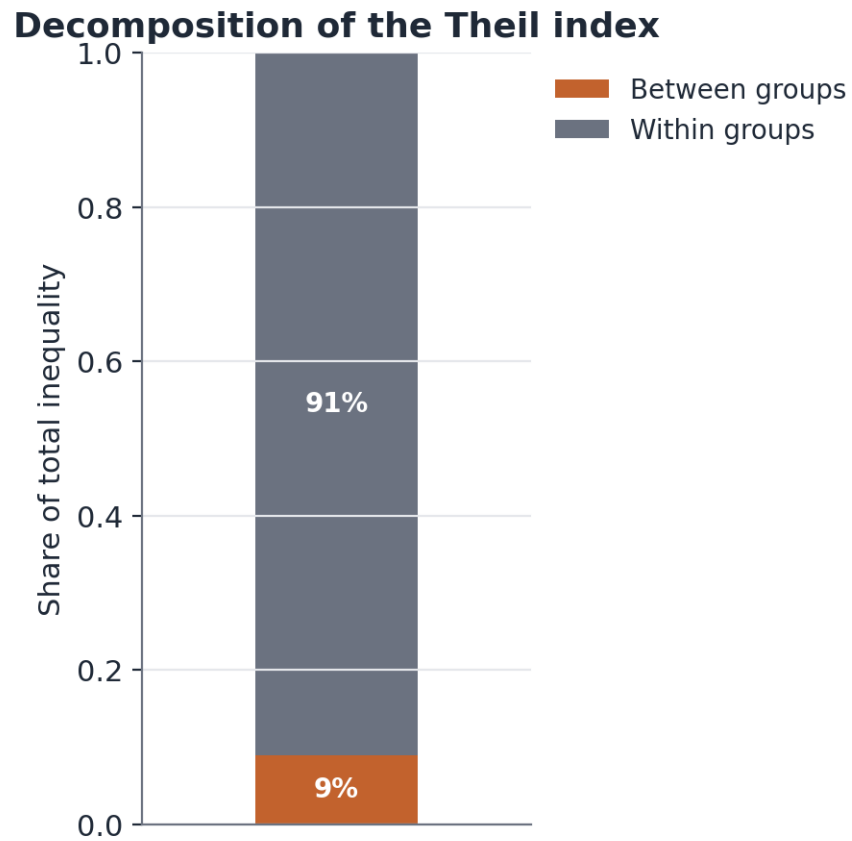
That sum isn’t an approximation — the two parts add up to exactly the total. The Gini index, by contrast, doesn’t share this simple form of exact decomposability — it’s computed separately for each group (or for the whole population), and it’s used to compare how concentrated the distribution is within each slice.

4. Easy conceptual example

Picture a company with two departments. If everyone in Department A earned exactly the same salary, and everyone in Department B also earned a fixed salary (just different from A’s), all of the company’s inequality would come from the difference between the two salaries — 100% “between groups,” 0% “within groups.” In practice, within any real department there’s also salary variation between people — and the Theil index separates exactly how much of the total inequality comes from each of these two sources.

5. How it works, broadly

1. Compute the Theil index for the whole population, a measure of income dispersion relative to the mean.
2. Compute the Theil index within each group separately (using only that group’s observations).



Illustrative data, not drawn from any real project.

Figure 4: Stacked bar showing the split of total Theil inequality between the “between-groups” share and the “within-groups” share

3. The “between-groups” share is computed as if every person within a group received their group’s average income — it captures only the variation between group means.
4. The “within-groups” share is the weighted average (weighted by each group’s share of total income) of the Theil indices computed within each group.
5. The two shares add up to exactly the total Theil index.
6. In parallel, the Gini index is computed within each group to compare concentration, and weighted quantiles are estimated within each group using each observation’s sampling weight.

6. What the result means

If the “between-groups” share is small relative to the total, that means most of the income inequality is spread within each group, not between them — even if the average difference between groups is large and statistically significant. The two things don’t contradict each other: a between-group gap can be real and important, and still represent a small fraction of total inequality, because the inequality within each group is also large.

7. How to interpret it

A common mistake is to conclude that “only 7% of the inequality comes from race, so racial inequality doesn’t matter much.” That conflates two different questions: (1) is the average gap between groups large and significant? and (2) what fraction of total income variance does that gap explain? The answer to the first question can be “yes, large and significant” even when the answer to the second is “a small fraction of the total” — because the inequality within each group, on its own, is already enormous. Contextualizing the size of the “between-groups” share matters, but it doesn’t cancel out the importance of the gap itself.

Similarly, a lower Gini in one group doesn’t necessarily mean a “better situation” — it can simply reflect a distribution more compressed near the bottom, meaning everyone in that group earns relatively similarly low amounts, which is different from “high income, well distributed.”

8. When it’s useful

When you want to contextualize the size of a between-group gap within the overall picture of inequality — for instance, showing that even though most income inequality sits within each racial group (not between them), the between-group gap is still large and statistically robust, and deserves attention on its own.

9. Important caveats

- The “between-groups” share tends to be small when the groups being compared are broad and internally heterogeneous (like large demographic categories) — that’s expected and shouldn’t be used to minimize the gap.
- The Theil index requires strictly positive values of the variable of interest (it doesn’t work directly with zero or negative income without adjustments).
- When comparing Gini indices across groups, always check whether the underlying distributions are comparable (same unit, same period), and remember a lower Gini isn’t synonymous with “better.”
- Weighted quantiles require the correct use of the survey’s sampling weights — ignoring the weights produces biased estimates when sampling isn’t simple random.

10. A small worked example

Suppose a hypothetical population split into two groups:

- Group A: average income = 4,000, internal Theil = 0.25.
- Group B: average income = 2,800, internal Theil = 0.30.
- Group A accounts for 40% of the population's total income; Group B, 60%.

The “between-groups” Theil index (computed from the difference in means, weighted by income share) comes out to about 0.04. The “within-groups” Theil index is the weighted average of the two internal Theils: $0.4 \times 0.25 + 0.6 \times 0.30 = 0.28$. Adding the two parts, total Theil is about 0.32.

This means the “between-groups” share accounts for roughly $0.04 / 0.32 = 12.5\%$ of total inequality — most of it (87.5%) comes from inequality that already exists within each group, even with a substantial difference in average income between the groups (4,000 vs. 2,800, a difference of over 40%).

11. Simple code example

```
import numpy as np

def theil_index(income):
    income = np.asarray(income, dtype=float)
    mean = income.mean()
    return np.mean((income / mean) * np.log(income / mean))

def gini_index(income):
    income = np.sort(np.asarray(income, dtype=float))
    n = len(income)
    indices = np.arange(1, n + 1)
    return (2 * np.sum(indices * income) - (n + 1) * np.sum(income)) / (n * np.sum(income))

# illustrative data
group_a = np.array([...]) # individual incomes, Group A
group_b = np.array([...]) # individual incomes, Group B

theil_total = theil_index(np.concatenate([group_a, group_b]))
theil_a = theil_index(group_a)
theil_b = theil_index(group_b)

share_a = group_a.sum() / (group_a.sum() + group_b.sum())
share_b = 1 - share_a

theil_within = share_a * theil_a + share_b * theil_b
theil_between = theil_total - theil_within

p90_a = np.average(np.quantile(group_a, 0.9)) # with sampling weights, use a weighted average
```

For surveys with a complex sampling design (weights, strata, clusters), libraries like `samplics` in

Python, or the **survey** package in R, compute weighted quantiles and inequality indices correctly, including their standard errors.

Duncan Dissimilarity Index

1. What problem does this solve?

Do two groups work across the same occupational categories, or is one group concentrated in certain roles while the other dominates different ones? A table with the percentage breakdown of each group across categories is hard to read at a glance — it's not obvious how unequal that split really is. The Duncan dissimilarity index (also called a segregation index) compresses that comparison into a single number between 0 and 1.

2. Intuition

Picture physically reshuffling people from two groups across a set of categories, with the goal of making the two percentage distributions identical. The Duncan index essentially measures **what fraction of one group would need to switch categories** for that to happen. If no reallocation is needed, the two groups are already distributed the same way. If nearly everyone in one group would need to move, the two groups are almost completely segregated from each other.

3. Plain explanation

For each category (say, each occupation), compute the share of Group A in that category and the share of Group B in the same category. Subtract one from the other, drop the sign (take the absolute value), and sum these differences across all categories. Divide by two. The result is the Duncan index: 0 means identical distributions across categories; 1 means complete segregation — no category is shared by both groups.

4. Simple conceptual example

Suppose two sales teams, A and B, are spread across four branches. If in every branch the share of A matches the share of B, the index is 0 — no segregation by branch. If instead A is entirely at the North branch and B is entirely at the South branch, the index is 1 — complete segregation. In practice the value almost always lands somewhere in between.

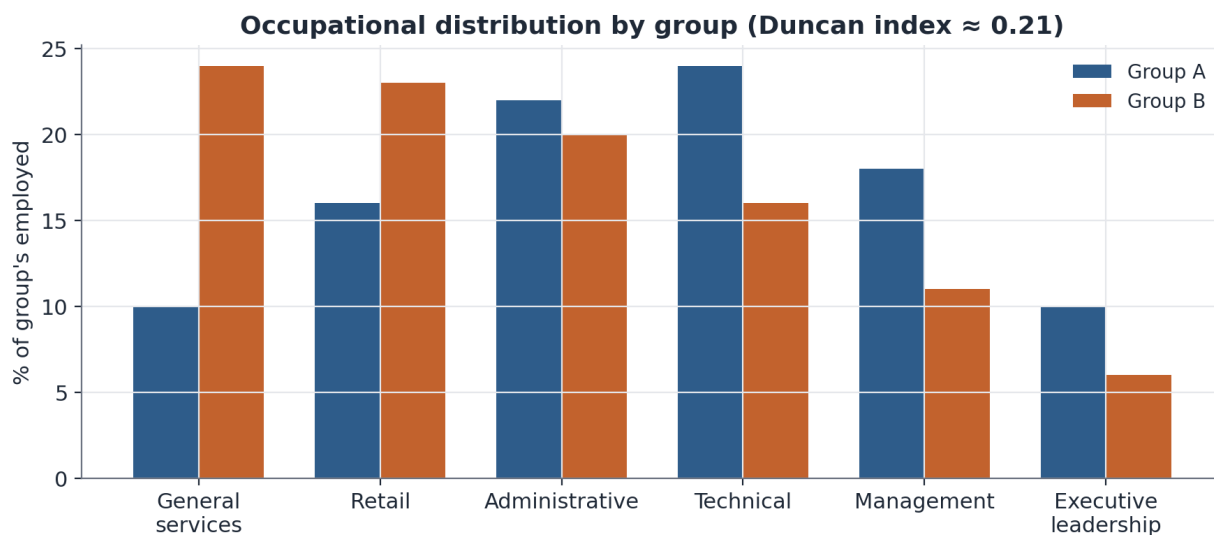
5. How it works, broadly

1. Define the categories that matter (occupation, branch, neighborhood — whatever fits the question).
2. Within each group, compute the share of people in each category — not the raw count, since the two groups may differ in overall size.
3. For each category, compute the absolute difference between the two shares.
4. Sum all the differences and divide by two.

Dividing by two avoids double-counting the same “distance” — without it, the index would range from 0 to 2, not 0 to 1.

6. What the result means

An index of 0.25, for instance, means that roughly 25% of one group would need to change categories to make the two distributions match. There is no universal threshold for “high” or “low” — the value only becomes meaningful when compared to another context (another country, another period, another pair of groups).



Illustrative data, not drawn from any real project.

Figure 5: Occupational distribution of two groups, with the resulting Duncan index

7. How to interpret it

The index describes **how unequal the distribution across categories is**, not **why** it is unequal. A high value is consistent with several different stories: real differences in qualifications between the groups, barriers to entry into certain categories, voluntary career choices, or some combination of all of these. The index alone does not separate these explanations.

8. When it's useful

It's the standard tool for measuring occupational, residential, or educational segregation between demographic groups — race, gender, region. In the Hub-Racial-Brasil project, an index of this kind measures how unevenly two racial groups are spread across occupational categories, complementing income-gap analyses: part of a wage gap can be tied to one group being concentrated in lower-paying occupations.

9. Important caveats

- The index depends directly on how the categories were defined. Grouping occupations more coarsely or more finely changes the result — only compare indices computed with the same classification.
- Categories with few observations make the share estimate unstable, artificially inflating or deflating the index.
- The index has no sign — it does not say which group comes out “ahead,” it only measures the distance between the two distributions.

10. A small worked example

Category	% of Group A	% of Group B	Absolute difference
Operations	15%	30%	15
Administrative	25%	25%	0
Technical	30%	25%	5
Management	30%	20%	10

Sum of absolute differences: $15 + 0 + 5 + 10 = 30$. Dividing by two: Duncan index = 15, or 0.15 on a 0-to-1 scale. That means 15% of one group would need to switch categories to equalize the two distributions — moderate segregation, concentrated mostly in the operations and management categories.

11. Simple code example

```
import numpy as np
import pandas as pd

# each group's share by category (each column should sum to 1)
data = pd.DataFrame({
    "category": ["operations", "administrative", "technical", "management"],
    "group_a": [0.15, 0.25, 0.30, 0.30],
    "group_b": [0.30, 0.25, 0.25, 0.20],
})

duncan_index = 0.5 * np.sum(np.abs(data["group_a"] - data["group_b"]))
print(f"Dissimilarity index: {duncan_index:.3f}")
```

Structural Break Test (Chow-Type)

1. What problem does this solve?

A series over time — a participation rate, an average price, a proportion — appears to have changed behavior at a specific, known moment: the start of a pandemic, a law taking effect, a documented economic crisis. Is that change real, or could it just be normal fluctuation in the series? The structural break test, in the tradition of the Chow test, answers this question specifically when the date of the possible event is already known before looking at the data.

2. Intuition

If nothing special had happened at that moment, the series' behavior before and after the chosen date should keep following, roughly, the same pattern — same trend, same relationship with other variables. The test fits a separate model to each of the two pieces of the series (before and after the date) and checks whether those two models are statistically different enough to reject the idea that they come from a single, continuous process.

3. Plain explanation

There are three ways to fit the same type of model: one for the entire series (ignoring the possible break), one for the period before the date, and a separate one for the period after. If the “with break” fit (two separate models) explains the data noticeably better than the “no break” fit (a single model), that's evidence something actually changed at the stated date.

4. Simple conceptual example

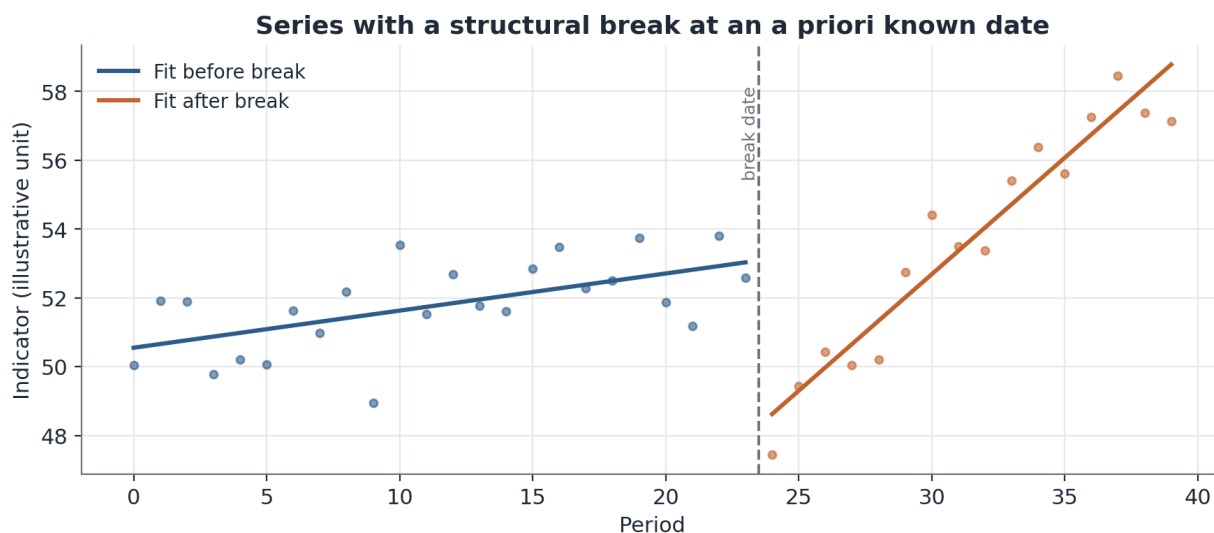
Picture a restaurant's daily order count over two years, with a known date when the menu was completely overhauled. Before the change, orders grew slowly and steadily. After it, they grew much faster. A structural break test at that specific date checks whether this pattern difference is too large to be sampling coincidence.

5. How it works, broadly

1. Choose the event date **before** looking at the results — the test loses its validity if the date is picked after observing where the series “looks like” it changed.
2. Fit a single model (say, a trend line) using the entire series.
3. Fit two separate models: one using only the data before the date, one using only the data after.
4. Compare how much the two separate models reduce the fitting error relative to the single model, using an F statistic.
5. A low p-value indicates the break improves the fit more than would be expected by chance — evidence of a structural change.

6. What the result means

A significant test says the series' behavior before and after the chosen date is statistically different. That's consistent with “something relevant happened at that moment” — but the test alone does not prove it was exactly the event you had in mind.



Illustrative data, not drawn from any real project.

Figure 6: Series with a structural break at an a priori known date

7. How to interpret it

The result is evidence of temporal correlation with the chosen date, not causation. If another relevant event happened close to the same date — a currency crisis near a labor-law reform, for instance — the test alone cannot attribute the change to one or the other. Contextual judgment, and ideally other sources of evidence, are needed to interpret the cause.

8. When it's useful

It's useful whenever there is a date of interest defined ahead of time, for reasons external to the data — a law, a policy, a historical event. In the Hub-Racial-Brasil project, this test is used to check whether a group's labor-market participation changed pattern at a specific, known moment, such as the start of a crisis or the implementation of a policy.

9. Important caveats

- This is a test for **one** break known a priori — not a search for multiple breaks at unknown dates (that's a different problem, addressed by methods such as Bai-Perron).
- Choosing the date after already seeing where the series “looks like” it breaks invalidates the test — the p-value no longer carries its usual interpretation.
- The test assumes a specific model form (typically linear) in each piece of the series; if the true relationship is very different from that, the result can mislead.
- Short segments before or after the date reduce the test's power to detect a real break that does exist.

10. A small worked example

A 40-observation series has, before the cutoff date, an estimated slope of 0.15 per period. After the cutoff, the estimated slope rises to 0.55 per period, and the series' average level also rises. Fitting the single model (no break) versus the two separate models, the resulting F statistic is 4.35, with

a p-value of 0.021 — below the conventional 0.05 threshold. Conclusion: there is evidence of a pattern change at the chosen date.

11. Simple code example

```
import numpy as np
import statsmodels.api as sm

# series: array with the variable of interest over time
# break_index: a priori known position where the break is tested
def chow_test(series, time, break_index):
    X_full = sm.add_constant(time)
    model_full = sm.OLS(series, X_full).fit()
    rss_full = model_full.ssr

    X1 = sm.add_constant(time[:break_index])
    X2 = sm.add_constant(time[break_index:])
    rss1 = sm.OLS(series[:break_index], X1).fit().ssr
    rss2 = sm.OLS(series[break_index:], X2).fit().ssr

    k = X_full.shape[1]
    n = len(series)
    f_stat = ((rss_full - (rss1 + rss2)) / k) / ((rss1 + rss2) / (n - 2 * k))
    return f_stat
```

Difference-in-Differences (DiD)

1. What problem does this solve?

A policy, a law, or an event affected a specific group, while a similar group was left out. How do you know whether the change observed in the affected group after the event is the result of the event itself, rather than just the continuation of a trend that was already happening anyway? The difference-in-differences (DiD) design uses the unaffected group as a reference to answer that question with a reading closer to causal than a simple before-and-after comparison.

2. Intuition

Comparing only the affected group before and after the event is misleading, because plenty of things change over time for reasons that have nothing to do with the event. The fix is to compare the **change** in the affected group with the **change** in the unaffected group, over the same period. If the two groups had been following similar trajectories before the event, the difference between these two changes can be attributed, more safely, to the event itself.

3. Plain explanation

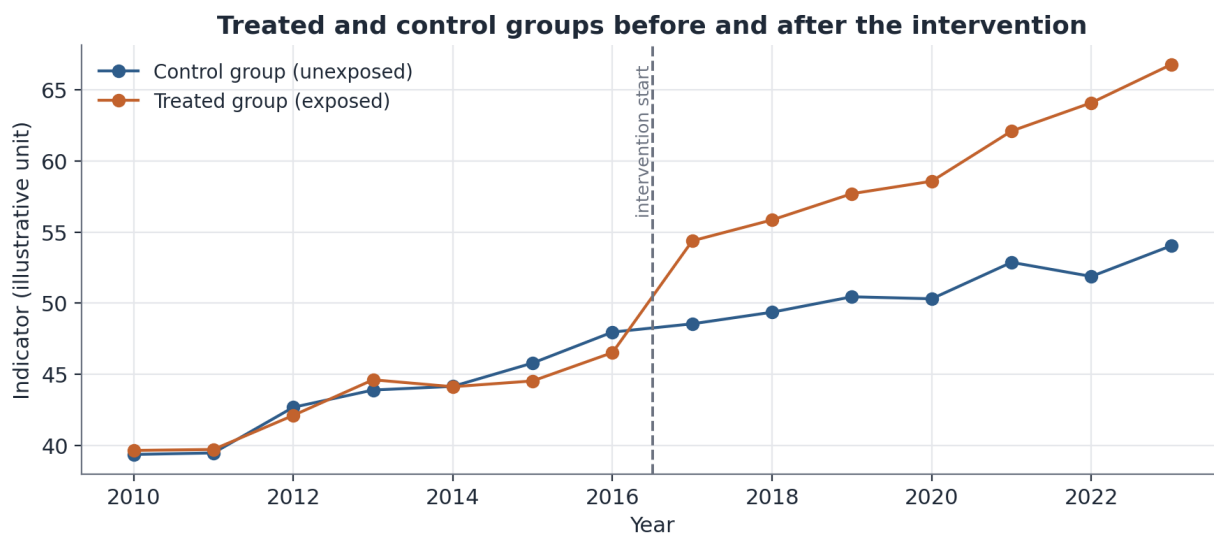
The name “difference in differences” comes exactly from that: you first compute the difference (change) within each group, then compute the difference between those two differences. This basic design can be extended in several ways: comparing **cohorts** defined by when they were born or entered some system (cohort design), tracking the effect **period by period** relative to the event (event-study), and testing whether the result holds up under **fake dates** (placebo) — an essential verification step, not an optional detail.

4. Simple conceptual example

Picture a city that introduced a new free public-transit program for students, while a similar neighboring city introduced nothing. Before the program, school attendance in both cities had been rising at a similar pace. After the program, attendance in the city that adopted it rose noticeably faster than in the neighboring city. The difference between these two changes is the estimated effect of the program.

5. How it works, broadly

1. Clearly define the treated group (affected by the event), the control group (unaffected), and the timing of the event.
2. Check the **parallel trends** assumption: were the two groups following similar trajectories before the event? This is done by comparing the pre-intervention periods, both visually and statistically.
3. Estimate the difference in average changes between the two groups, from the pre- to the post-event period.
4. When several periods of data are available, an **event-study** estimates the effect separately for each period relative to the event, instead of a single aggregate number — this reveals whether the effect appears abruptly, grows over time, or was already present before the event (which would weaken the parallel trends assumption).
5. Run **placebo** tests: apply the same design to a fake date, with no real policy change, and check whether an “effect” shows up anyway. If it does, that’s a warning sign about the design.



Illustrative data, not drawn from any real project.

Figure 7: Treated and control groups before and after the intervention

6. What the result means

The estimated effect is the difference between the treated group's change and the control group's change, over the same period. With good pre-event parallel trends and no effect showing up in the placebo tests, this difference carries a reasonably defensible causal reading — stronger than comparing it against a simple single-group before-and-after series.

7. How to interpret it

DiD gives a **stronger** causal reading than comparing before and after for a single group, but it still **depends** on the parallel-trends assumption — that is, on the idea that, absent the event, the two groups would have kept following similar trajectories. That assumption is not directly testable for the post-event period (that's precisely what we don't observe); what can be tested is whether it held during the pre-event period. "Parallel trends holding up in the pre-period" is not proof that the assumption is true — it's only evidence that the data doesn't contradict it, which is a considerably weaker reading.

8. When it's useful

It's useful whenever there's a policy or event with a clearly affected group and a clearly unaffected one, with data available before and after. In the Hub-Racial-Brasil project, this design is used to explore whether a law — such as a racial quota policy — had a causal effect, comparing cohorts born before and after a cutoff date, or sectors affected and unaffected by the law over time.

9. Important caveats

- "Finding no effect" is not the same as "there is no effect" — a design with few groups, few periods, or a very noisy metric may simply lack the statistical power to detect a real effect.
- The choice of control group matters a great deal: a group that's poorly comparable to the treated group weakens the parallel-trends assumption.

- Always report placebo results alongside the headline effect — reporting only the favorable result is a common (and misleading) way to present DiD.

10. A small worked example

Before the event, the treated group's indicator rises from 40 to 47 (a change of +7); the control group rises from 40 to 46 (a change of +6) — very similar trends. After the event, the treated group rises from 47 to 68 (a change of +21); the control group rises from 46 to 53 (a change of +7). The DiD estimate is $(21 - 7) = 14$ units of effect attributable to the event, above what was already expected from the trend common to both groups.

11. Simple code example

```
import pandas as pd
import statsmodels.formula.api as smf

# panel_data: columns [unit, period, treated, post, outcome]
# treated: 1 if belongs to the treated group, 0 otherwise
# post: 1 if period is after the event, 0 otherwise
model = smf.ols("outcome ~ treated * post", data=panel_data).fit(
    cov_type="cluster", cov_kwds={"groups": panel_data["unit"]}
)
print(model.params["treated:post"]) # DiD effect estimate
print(model.pvalues["treated:post"])
```

Rotating Panel Transition Analysis

1. What problem does this solve?

Comparing two isolated snapshots — a group’s unemployment rate this quarter and the same group’s rate next quarter — tells you nothing about what happened to people individually. Were the same people still unemployed, or did one wave leave unemployment while another entered, keeping the aggregate rate looking similar? Rotating panel transition analysis answers that question by tracking individuals across successive interviews, rather than just comparing aggregate snapshots.

2. Intuition

Some household surveys interview the same household (and, to the extent possible, the same residents) multiple times over a period — that’s the “rotating panel”: part of the sample is replaced each round, but another part stays on, making it possible to observe the same person across consecutive interviews. This lets you ask directly: of those who were unemployed in the first interview, what fraction were employed in the next one? And does that fraction differ across groups?

3. Plain explanation

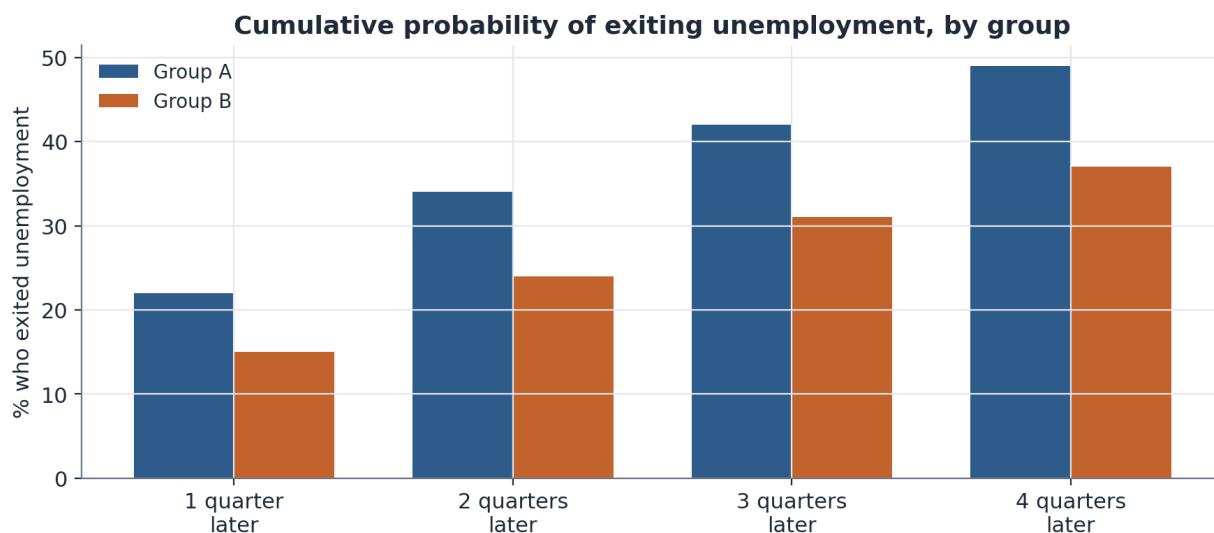
After matching records for the same person across consecutive interviews, you build a **transition matrix**: rows represent the starting state (say, unemployed), columns represent the following state (say, formal employment, informal employment, unemployed, out of the labor force), and each cell shows the share of people who made that specific transition. Comparing this matrix across groups reveals mobility differences that comparing aggregate rates would not capture.

4. Simple conceptual example

Picture two groups of subscribers to a service, both with the same aggregate cancellation rate in a given month. In Group A, almost everyone who canceled had already been signaling dissatisfaction for months. In Group B, the cancellations come from recently joined customers who never actually started using the service. The aggregate rate is the same, but the story behind it — and the recommended action — is completely different. Tracking individual transitions is what reveals that difference.

5. How it works, broadly

1. Identify, in the dataset, the records that correspond to the same person across consecutive interviews — typically via a household identifier combined with characteristics that help confirm it’s the same person (age, sex, relationship to the household head).
2. Discard or handle separately the cases where matching fails — people who moved households, households with resident turnover, or inconsistent records.
3. For each pair of consecutive interviews, classify the person’s state at each moment (employed, unemployed, out of the labor force, for example).
4. Build the transition matrix, separately for each group of interest.
5. Compare specific transition rates (say, the probability of exiting unemployment) across groups, with proper significance testing.



Illustrative data, not drawn from any real project.

Figure 8: Cumulative probability of exiting unemployment, by group

6. What the result means

A higher transition rate for one group — say, exiting unemployment more often — indicates greater observed real mobility for that group within this specific panel, something a comparison of aggregate rates between two isolated snapshots could not distinguish from mere compositional turnover.

7. How to interpret it

The transition rate describes what happened to the people who were successfully matched between interviews — not the whole population. If matching fails systematically differently across groups (say, one group has higher rates of moving households, and is therefore harder to track), the estimated transition rates may not correctly represent the experience of the entire group, only of the fraction that remained trackable.

8. When it's useful

It's the right tool whenever the question is about individual mobility — transitions between employment and unemployment, between formal and informal work, between income bands — and the survey used has a rotating panel design (the same unit interviewed more than once). In the Hub-Racial-Brasil project, this kind of analysis uses the fact that the same person appears in a few consecutive interviews of a household survey to track individual transitions and compare mobility rates across racial groups.

9. Important caveats

- A rotating panel is **not** the same as a full longitudinal panel, in which the entire same sample is followed for many years — in a rotating panel, each person is only observed for a limited number of rounds before leaving the sample.
- Much of a rotating panel's data is actually repeated cross-section outside the window in which matching is possible — it's essential to know exactly which rounds allow tracking the same

person.

- Matching between interviews can fail for several reasons: a household move, resident replacement, typos in identifiers, or plain sample attrition (the person can no longer be located). The matching success rate should always be reported.
- If the matching success rate differs between the groups being compared, this can introduce selection bias into the results — it's worth testing for this explicitly.

10. A small worked example

Out of 500 unemployed people in Group A in the first interview, 420 (84%) could be matched to a second interview one quarter later; of those, 92 were employed — a transition rate of 21.9%. Out of 480 unemployed people in Group B, 390 (81%) were matched; of those, 59 were employed — a rate of 15.1%. The nearly 7-percentage-point gap is tested statistically (say, with a proportions test) to check whether it's too large to be attributed to chance.

11. Simple code example

```
import pandas as pd
from statsmodels.stats.proportion import proportions_ztest

# matched_data: one row per successfully matched person,
# columns: group, state_t0, state_t1
unemployed_t0 = matched_data[matched_data["state_t0"] == "unemployed"]

transition_by_group = unemployed_t0.groupby("group")["state_t1"].apply(
    lambda s: (s == "employed").mean()
)
print(transition_by_group)

counts = unemployed_t0.groupby("group")["state_t1"].apply(
    lambda s: (s == "employed").sum()
)
totals = unemployed_t0.groupby("group").size()
statistic, p_value = proportions_ztest(counts.values, totals.values)
print(statistic, p_value)
```

A/B testing fundamentals

1. What problem does this solve?

A product team changes something — a button’s copy, the order of two signup steps, a new recommendation algorithm — and wants to know whether it actually improves the metric that matters, or whether any change observed after launch is just day-to-day noise. Comparing “before” and “after” doesn’t settle this on its own, because plenty of things besides the change under test also move over time (seasonality, a marketing push, a holiday). An A/B test settles it by comparing two versions at the same time, in groups of users formed by random assignment.

2. Intuition

If you randomly split a large group of users into two, the two subgroups tend to look alike on everything — age, device, purchase habits, how good a day they’re having — except for the one thing you deliberately made different between them: which version of the product each one saw. If the average outcome then differs between the groups, the simplest explanation is the change itself, not some pre-existing difference between the users. That logic — simultaneous comparison with random assignment — is what makes an A/B test a more trustworthy read than comparing two different time periods.

3. Plain explanation

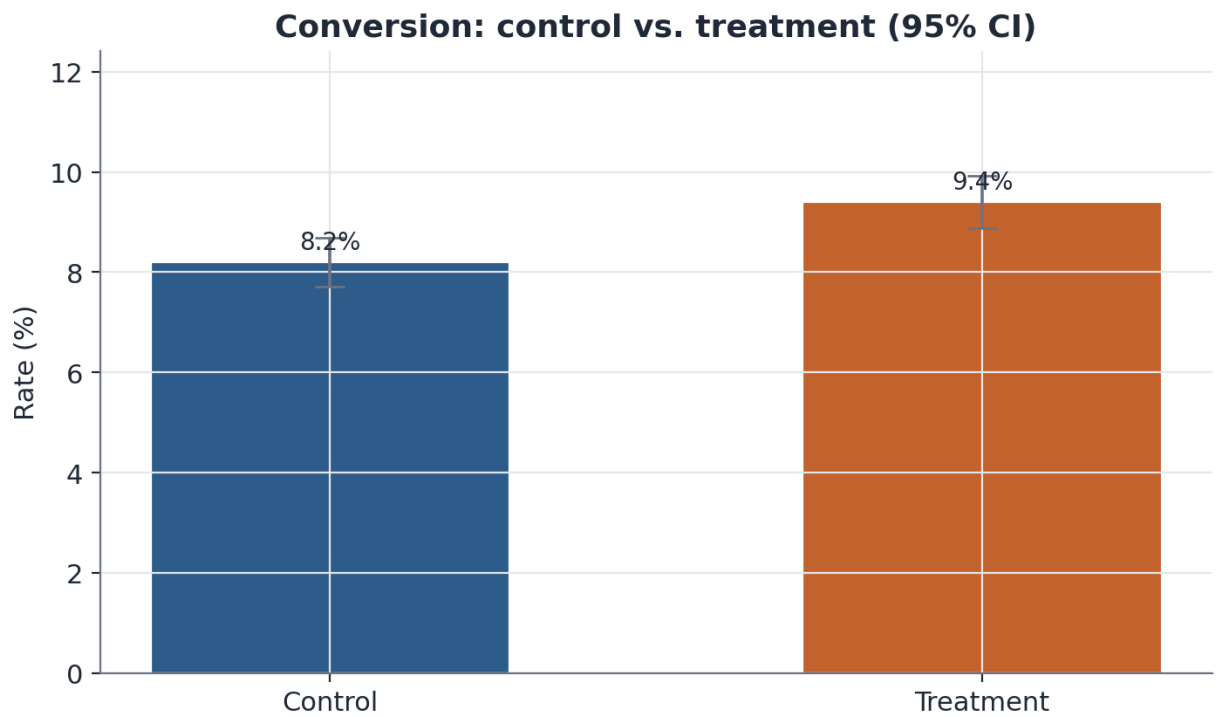
After the experiment runs long enough, you compute the difference between the treatment group’s average metric (or proportion) and the control group’s. That observed difference is almost never exactly zero, even when the change has no real effect at all — there’s always sampling variation. The hypothesis test asks: is this difference large enough, given the group sizes and the metric’s variability, that it’s implausible to explain by chance alone? The 95% confidence interval around the difference shows the range of plausible values for the true effect, not just a single number.

4. A simple conceptual example

Picture a site testing a new “checkout” button color. Half of visitors, chosen at random, see the usual blue button; the other half see a new green one. After two weeks, you compare the conversion rate between the two groups. Because the split was random, any consistent difference large enough between the groups is likely to come from the button color — not from one group happening, by coincidence, to contain more people already inclined to buy.

5. How it works, broadly

1. Before running the experiment, define a primary metric (say, conversion rate) and the sample size needed.
2. Randomly assign each eligible user to control or treatment, usually in a fixed split (50/50 is common, but not required).
3. Run the experiment for a pre-defined period, without repeatedly peeking at the result and making decisions along the way.
4. Compute the difference between the two groups’ means (or proportions), the standard error of that difference, and from there the test statistic, the p-value, and the 95% confidence interval.



Illustrative data, not drawn from any real project.

Figure 9: Bar chart of conversion for the control and treatment groups, each with its 95% confidence interval

5. Before trusting the result, check whether randomization actually worked as planned — this is where complementary checks such as a Sample Ratio Mismatch test and a covariate balance check come in.

6. What the result means

A low p-value (typically below 0.05) indicates a difference this large would be unusual if the change truly had no effect — evidence in favor of a real effect. The confidence interval shows the plausible magnitude of that effect: a narrow interval sitting away from zero suggests a real, fairly well-estimated effect; a wide interval, even one that excludes zero, means the exact size of the effect is still uncertain.

7. How to interpret it

Statistical significance is not the same as practical importance: a 0.1 percentage-point effect can be statistically significant with a large enough sample, yet irrelevant to the business decision. And “not significant” is not the same as “no effect” — it can simply mean the experiment lacked enough sample to detect an effect of the size that actually exists. Always read the effect size and its confidence interval alongside the p-value, never the p-value on its own.

8. When it’s useful

Whenever it’s possible to randomize who receives each version of a product, process, or message, and there is a clear primary metric to judge the outcome by — typical of interface tests, recommendation algorithms, pricing policies, or signup flows. These fundamentals — effect size, standard error, 95% CI, hypothesis testing — are the starting point for nearly every more advanced experiment reading, including randomization-integrity checks, covariate adjustment, and non-inferiority testing.

9. Important caveats

- Validity depends on randomization actually having worked — never assume that without checking (see Sample Ratio Mismatch).
- Repeatedly peeking at results while the experiment runs and stopping as soon as it “turns significant” inflates the false-positive rate; fix the sample size and the run period before starting.
- Testing many secondary metrics at once raises the odds of finding something “significant” by chance; treat secondary findings as hypotheses for the next experiment, not as conclusions.
- A statistically significant win on one metric can come paired with a loss on another, guardrail metric — that’s exactly what non-inferiority testing exists to check.

10. A small worked example

A site tests a new checkout button against the current one, with 12,000 visitors in each group. The control group converts at 8.2%; the treatment group converts at 9.4% — a difference of 1.2 percentage points, or roughly a 15% relative lift. The standard error of the difference, computed from each group’s size and the variability of a conversion rate, yields a z-statistic of about 3.0 and a p-value below 0.01. The 95% confidence interval for the difference runs roughly from 0.4 to 2.0 percentage points — entirely above zero, reinforcing that the gain is unlikely to be sampling noise alone.

11. Simple code example

```
import numpy as np
from statsmodels.stats.proportion import proportions_ztest, confint_proportions_2indep

conversions = np.array([984, 1128])    # illustrative: control, treatment
visitors = np.array([12000, 12000])    # illustrative

statistic, p_value = proportions_ztest(conversions, visitors)
ci_low, ci_high = confint_proportions_2indep(
    conversions[1], visitors[1], conversions[0], visitors[0], method="wald"
)

print(f"z = {statistic:.2f}, p-value = {p_value:.4f}")
print(f"95% CI for the difference: [{ci_low:.4f}, {ci_high:.4f}]")
```

The two-proportion test collapses, into a single statistic, whether the observed gap between the two groups' conversion rates is too large to be explained by sampling variation alone.

Sample Ratio Mismatch (SRM) test

Assumes the A/B testing fundamentals — see [A/B testing fundamentals](#) for the vocabulary of randomization, groups, and primary metric used here.

1. What problem does this solve?

An experiment was planned with a specific split between arms — 50/50 is the most common, but it could be 90/10, 70/30, or any other proportion set in advance. After running it, did that split actually come out as planned? If the randomization system has a bug, if some type of user is being dropped differently between groups, or if an eligibility filter interacts badly with assignment, the real split can come out skewed — and when that happens, nothing the experiment measures afterward can be trusted.

2. Intuition

If you flip a fair coin 20,000 times, you don't expect exactly 10,000 heads and 10,000 tails — but you do expect something close to it, with a deviation that follows a predictable pattern. A Sample Ratio Mismatch happens when the deviation from the planned proportion is too large to be explained by that expected chance variation. It's essentially asking: "is this coin still fair, or is something broken with it?" — except here the coin is the experiment's own randomization mechanism.

3. Plain explanation

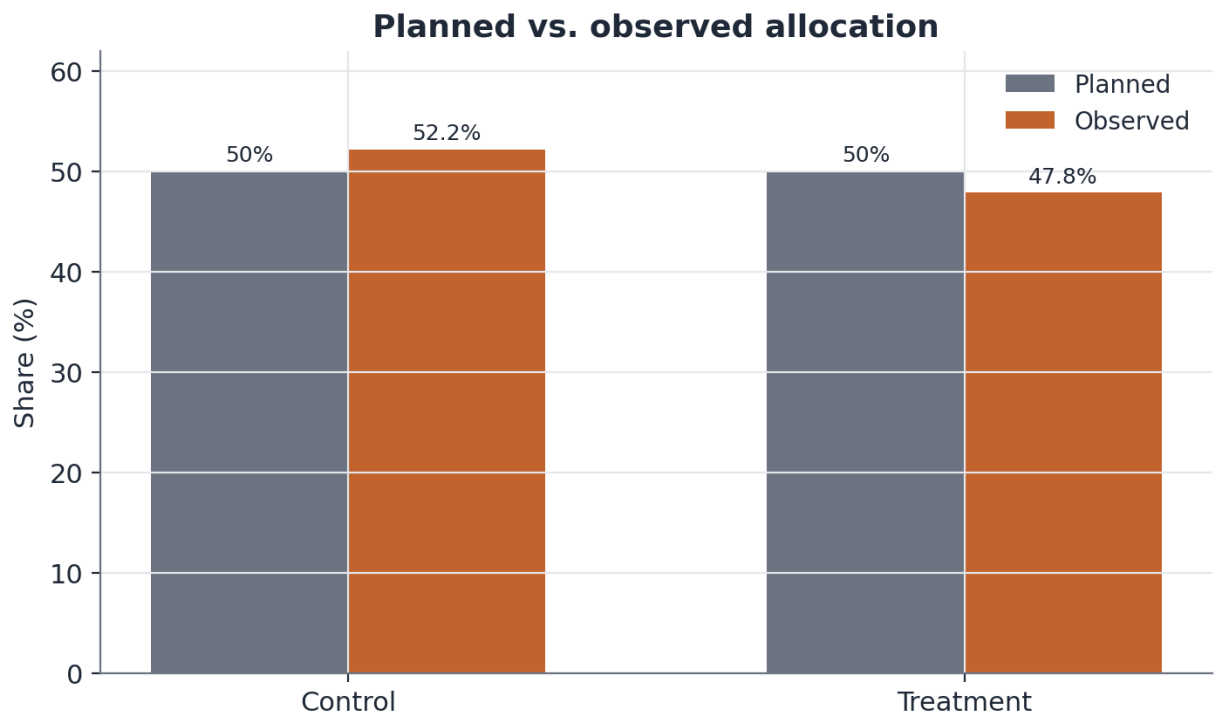
The test compares the observed user count in each arm of the experiment against the count you'd expect under the planned proportion, using a chi-square test (or, equivalently for two groups, a binomial test). The larger the gap between observed and expected, the stronger the evidence that randomization isn't behaving as it should.

4. A simple conceptual example

An app plans to test a new onboarding screen for 50% of new users, keeping the other 50% on the current screen. By the end of the experiment, the current-screen group has 10,432 users and the new-screen group has 9,568 — a gap of nearly 900 users. That could just be normal fluctuation, or it could signal that, for some technical reason, users on a certain device type are being excluded from the new group before they're even counted. The SRM test helps decide which of the two scenarios is more plausible.

5. How it works, broadly

1. Define the planned allocation proportion between arms (e.g., 50%/50%).
2. Count how many users actually landed in each arm by the end of the experiment (or of a checkpoint period).
3. Compute the expected count in each arm by applying the planned proportion to the observed total.
4. Run a chi-square goodness-of-fit test comparing observed and expected counts, producing a p-value.
5. A very low p-value (the common threshold is much stricter than for an effect test — something like 0.001, because a false alarm here halts the entire experiment) is the warning sign.



Illustrative data, not drawn from any real project.

Figure 10: Bar chart comparing the planned proportion and the observed proportion for each experiment arm

6. What the result means

A non-significant SRM doesn't prove randomization is perfect — only that the observed proportion is compatible with the planned one, within what's expected from random variation. A significant SRM is evidence that the real group proportions deviated from the planned ones by more than chance would suggest — a sign that something in the assignment mechanism, or in how users are counted, isn't working correctly.

7. How to interpret it

A significant SRM doesn't say, on its own, what caused the imbalance — only that there's a problem to investigate before trusting any effect reading from the experiment. Common causes include: bugs in the assignment code, differences in load time between versions (causing one to “lose” more users to timeouts), bot or traffic-quality filters applied asymmetrically, and logging issues that record one group more completely than the other.

8. When it's useful

In every randomized experiment, as a routine check before interpreting any effect result — it's the first test to run, not an optional extra. It matters especially in experiments with many technical systems involved in assignment (multiple platforms, multiple entry points, complex eligibility filters), where the odds of a subtle bug breaking randomization are higher.

9. Important caveats

- A significant SRM invalidates any measured effect until the cause is found and fixed — there's no point “looking at the effect anyway” or trying to statistically compensate for the imbalance afterward.
- The significance threshold used is usually stricter than the standard 0.05 (something like 0.001 or lower), because the cost of ignoring a real SRM is high and the check itself is cheap to run.
- No SRM in the overall allocation doesn't guarantee no SRM within specific subgroups (a device type, a region) — checking by segment is worthwhile when the suspected cause is localized.
- SRM can show up only during part of the experiment's run (e.g., a bug fixed midway) — looking at the proportion over time, not just the cumulative total, helps diagnose that.

10. A small worked example

An experiment was planned with a 50/50 split. By the end, the control group has 10,432 users and the treatment group has 9,568, out of a total of 20,000. The expected value in each arm, under the planned proportion, is 10,000. The chi-square statistic is:

$$\chi^2 = \frac{(10,432 - 10,000)^2}{10,000} + \frac{(9,568 - 10,000)^2}{10,000} \approx 37.3$$

With 1 degree of freedom, that value corresponds to an extremely low p-value (well below 0.001) — strong evidence that the observed proportion isn't compatible with a random 50/50 split. Before interpreting any effect measured in this experiment, the cause of that discrepancy needs to be found.

11. Simple code example

```
from scipy.stats import chisquare

observed = [10432, 9568]          # illustrative
planned_share = [0.5, 0.5]
total = sum(observed)
expected = [p * total for p in planned_share]

result = chisquare(f_obs=observed, f_exp=expected)
print(f"chi-square = {result.statistic:.2f}, p-value = {result.pvalue:.6f}")
```

A p-value below the strict SRM threshold (say, 0.001) is the signal to stop and investigate before moving ahead with any effect analysis.

Covariate balance and covariate adjustment

Assumes the A/B testing fundamentals — see [A/B testing fundamentals](#).

1. What problem does this solve?

An experiment was randomized, but did the two groups actually come out alike on characteristics that existed before the experiment started — user age, prior product tenure, historical behavior? And, separately: given that such pre-experiment characteristics were measured, can they be used to make the effect estimate more precise, even when the groups are already well balanced? These are two related but different questions, and they’re commonly conflated.

2. Intuition

A balance check is an audit: with groups formed by random assignment, characteristics that shouldn’t have any relationship to the assignment — because they were measured before the experiment even existed — should be distributed similarly across groups. If they’re very different, that’s a red flag about randomization, not a normal feature of the experimental design.

Covariate adjustment is a different thing: even with perfect randomization, there’s always some user-to-user variation that has nothing to do with the treatment. If a characteristic measured before the experiment (like last month’s purchase value) helps predict the metric under test, “netting out” that characteristic from the comparison reduces the estimate’s noise — without biasing the result, because the characteristic was measured before any treatment effect could exist.

3. Plain explanation

For balance: compare, for each covariate of interest, the mean (or proportion) between the control and treatment groups, typically through the standardized mean difference — the difference divided by a pooled standard deviation, which lets you compare covariates on very different scales (age in years, spend in dollars) on a common ruler.

For adjustment: instead of comparing only the two groups’ raw averages, fit a model that already accounts for these pre-experiment covariates (a regression, for instance), or use post-stratification — splitting the sample into strata defined by the covariates and combining the estimates within each stratum. The result is an estimate of the same effect, but with a smaller standard error.

4. A simple conceptual example

A subscription app tests a new pricing policy. Before looking at any experiment outcome, the team compares average age, prior subscription tenure, and average spend from the month before the experiment between the control and treatment groups. All three are close between groups — a sign that randomization worked. Later, when estimating the pricing policy’s effect on revenue, the team uses last month’s spend as an adjustment covariate, because they know past spend is a good predictor of future spend — this makes the effect estimate more precise, with a narrower confidence interval, without changing its meaning.

5. How it works, broadly

Balance (pre-experiment check): 1. List the relevant covariates, measured before the experiment started (never afterward — post-treatment covariates may already have been affected by the

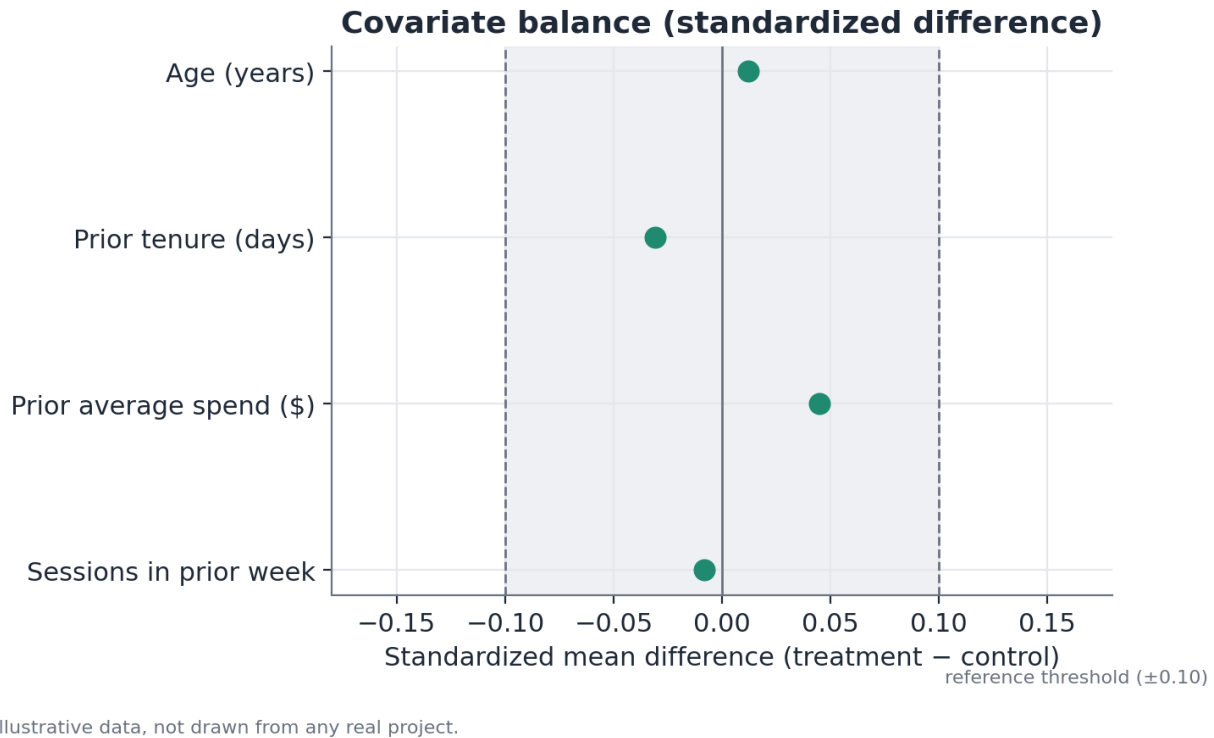


Figure 11: Balance chart (love plot) showing each covariate’s standardized difference between groups, with a reference band around zero

experiment itself). 2. Compute the standardized mean difference between groups for each covariate. 3. Compare each difference to a common reference threshold (0.1 is a commonly used value in the literature) — below that, balance is considered acceptable.

Covariate adjustment (variance reduction): 1. Choose pre-experiment covariates that are good predictors of the metric of interest. 2. Fit a model (a simple linear regression, or a technique like CUPED) that uses those covariates to explain part of the metric’s variation that has nothing to do with the treatment. 3. The treatment-effect estimate, after this adjustment, has the same interpretation as before, but with reduced standard error.

6. What the result means

Acceptable balance across all observed covariates gives more confidence that the groups were comparable before the experiment started — reinforcing the causal reading of the effect test. Covariate adjustment, on the other hand, doesn’t change the interpretation of the estimated effect (it’s still the difference caused by the treatment); it only reduces the uncertainty around that estimate, allowing smaller effects to be detected with the same sample, or the same precision to be reached with a smaller sample.

7. How to interpret it

An imbalance in one covariate doesn’t automatically invalidate the experiment, but it warrants investigation — it might just be a false positive (with many covariates tested, some will look imbalanced by chance) or it could signal a real randomization problem, in which case a complementary

Sample Ratio Mismatch test is the natural next step. Good balance on observed covariates never guarantees balance on unobserved ones — it’s evidence in favor of randomization, not complete proof.

8. When it’s useful

The balance check should be routine in every randomized experiment, as part of validating that randomization worked. Covariate adjustment is especially valuable when the primary metric has high variability across users (common, for instance, in revenue or engagement metrics) and there’s a pre-experiment covariate strongly correlated with it — in those cases, the precision gain can be substantial.

9. Important caveats

- Never use a covariate measured after the experiment started for adjustment or balance checking — it may already have been affected by the treatment itself, which introduces bias instead of reducing noise.
- Covariate adjustment doesn’t “fix” a real assignment imbalance — it reduces variance once randomization has already worked, not a substitute for broken randomization.
- Testing balance on dozens of covariates at once raises the odds of finding some “imbalanced” one by chance — look at the overall pattern, don’t react in isolation to a single covariate outside the threshold.
- Which covariates to use for adjustment should be decided before looking at the experiment’s results, to avoid retroactive selection bias.

10. A small worked example

A subscription app runs an experiment with 8,000 users per arm. Comparing average spend in the month before the experiment: control \$42.10 (standard deviation \$18.40), treatment \$42.80 (standard deviation \$18.90). The standardized mean difference is:

$$SMD = \frac{42.80 - 42.10}{\sqrt{(18.40^2 + 18.90^2)/2}} \approx 0.037$$

Well below the 0.1 reference threshold — acceptable balance. When estimating the pricing policy’s effect on next month’s revenue, the simple estimator (difference in means) produces a 95% CI with a half-width of ± 1.05 percentage points around the estimated effect; using last month’s spend as an adjustment covariate, the CI narrows to roughly ± 0.62 percentage points — the same estimated effect, with substantially more precision.

11. Simple code example

```
import numpy as np
import statsmodels.api as sm

# illustrative: y = revenue in the experiment month, treat = 1 if treatment
# prior_spend = pre-experiment covariate (spend in the prior month)
X_adjusted = sm.add_constant(np.column_stack([treat, prior_spend]))
model = sm.OLS(y, X_adjusted).fit(cov_type="HC1")
```

```
print(model.summary())  
# The 'treat' coefficient is the adjusted effect; its standard error tends  
# to be smaller than in a regression without the covariate, when it's  
# predictive.
```

The coefficient on the treatment indicator is the adjusted effect estimate; comparing its standard error to that of a regression without the covariate directly shows the precision gain achieved.

Non-inferiority testing

Assumes the A/B testing fundamentals — see [A/B testing fundamentals](#).

1. What problem does this solve?

Not every metric in an experiment is “bigger is better.” Some are guardrail metrics: things that must not get worse beyond a certain point, even if the primary metric improves a lot — fraud rate, load time, error rate, support complaints. The question there isn’t “did this metric change?”, it’s more specific: “did this metric not get worse than I can tolerate?” A standard two-sided test, built to ask whether there’s any difference at all, doesn’t answer that question well — because it treats “improved a lot” and “didn’t get much worse” the same way, when in practice only the second is what matters for a guardrail metric.

2. Intuition

Think of the difference between two kinds of question. A standard test asks: “is this metric different from that one?” — in either direction. A non-inferiority test asks something narrower: “is this metric not worse than that one by more than X?” — in one direction only, explicitly allowing it to be a little worse, up to a limit defined as acceptable beforehand. That margin X isn’t a number statistics picks for you; it’s a business decision, made before the experiment, about how much of a hit on this metric is tolerable in exchange for what’s gained elsewhere.

3. Plain explanation

Instead of testing whether the difference is exactly zero, you test whether the difference is smaller than the agreed non-inferiority margin — usually by building the confidence interval for the difference and checking whether it falls entirely below that margin (for a metric where “higher is worse,” like a fraud rate). If the confidence interval’s upper bound sits below the margin, there’s evidence of non-inferiority. If the interval crosses the margin, there isn’t enough evidence — which doesn’t necessarily mean the metric got much worse, only that the possibility can’t be confidently ruled out.

4. A simple conceptual example

A company launches a new fraud-detection model that’s faster, but with a small chance of being slightly less precise. The risk team decides, before the experiment, that an increase of up to 0.5 percentage points in the fraud rate is acceptable given the speed gain. The experiment runs, and the observed difference in fraud rate between the new model and the current one is small, with a confidence interval that falls entirely below 0.5 percentage points. That’s evidence of non-inferiority: even without proving the new model is exactly equal or better, there’s enough confidence that it doesn’t push fraud beyond what was defined as tolerable.

5. How it works, broadly

1. Set the non-inferiority margin before the experiment — a business decision, not a statistical one, about how much of a hit is tolerable.
2. Formulate the hypotheses one-sidedly: the null hypothesis is that the metric got worse than the margin; the alternative is that it did not get worse than the margin.

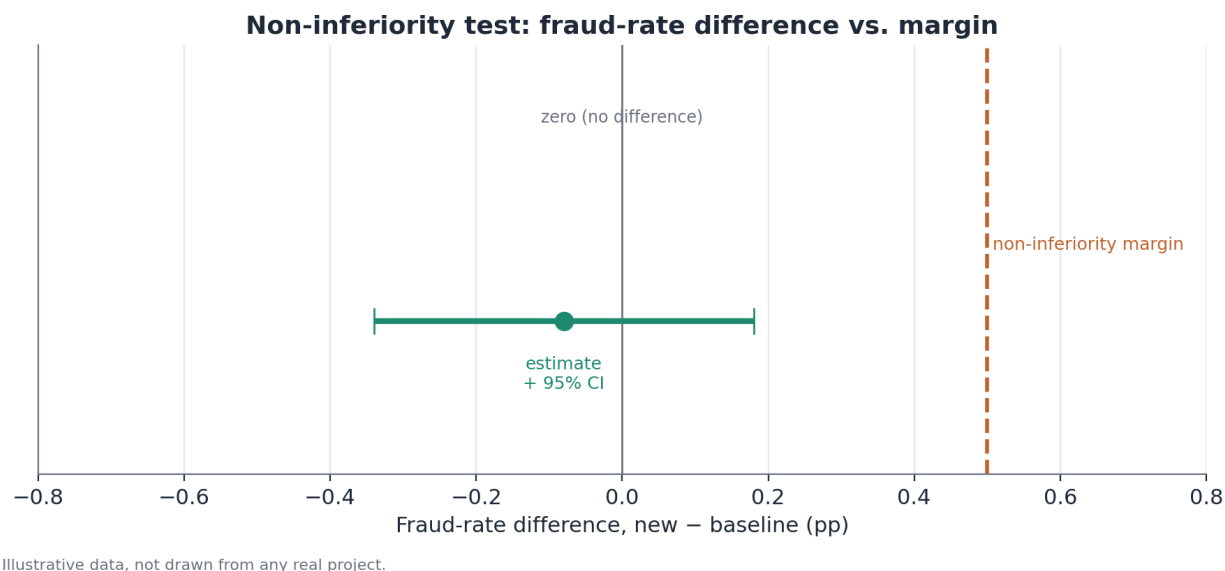


Figure 12: Point estimate and confidence interval for the fraud-rate difference, compared against the non-inferiority margin

3. Run the experiment as usual, collecting the observed difference and its confidence interval, typically at a one-sided confidence level (e.g., a one-sided 95%, equivalent to a two-sided 90% CI).
4. Compare the relevant bound of the confidence interval (the one that signals “worse”) against the defined margin.
5. If that bound falls within the acceptable margin, conclude non-inferiority.

6. What the result means

Concluding non-inferiority means there’s statistical evidence the metric didn’t get worse beyond the defined margin — it does not mean the metric is equal, nor that it improved. An inconclusive result (when the confidence interval crosses the margin) simply means the sample didn’t provide enough evidence to rule out a worse-than-margin deterioration — that’s not the same as concluding the metric actually got worse.

7. How to interpret it

The non-inferiority margin needs to be set before looking at the experiment’s data, and it should reflect an explicit business judgment: how much of this metric the organization is willing to “spend” in exchange for some other gain. There’s no “statistically correct” margin — it’s a choice, and different margins lead to different conclusions about the same data. It’s also worth remembering that “non-inferior” isn’t a synonym for “costless” — even within the margin, a small, consistent deterioration can add up over time or at scale.

8. When it’s useful

In any experiment with a guardrail metric that has a clear tolerance limit — safety, risk, technical performance, a minimum customer satisfaction bar. It’s commonly used alongside a traditional

effect test on the primary metric: the primary metric gets tested the usual way (is there a difference?), while guardrail metrics get tested for non-inferiority (is any deterioration within what's tolerable?).

9. Important caveats

- The margin needs to be fixed before the experiment — choosing it after seeing the result, to “make the test pass,” invalidates the test’s interpretation.
- A non-inferiority result is not the same as “the metric didn’t change” — it might have gotten slightly worse, just within what was deemed acceptable.
- A small sample tends to produce wide confidence intervals that cross the margin even when the true effect is small — an inconclusive result calls for attention to the design’s statistical power, not a hasty conclusion that the metric got worse.
- Check the margin’s direction carefully: for metrics where “higher is worse” (fraud, errors), the margin caps how much the metric can rise; for metrics where “lower is worse” (satisfaction, retention), the logic flips.

10. A small worked example

A company tests a new fraud-detection model against the current one, with a non-inferiority margin set at 0.5 percentage points (an increase in fraud rate larger than that would be unacceptable). The observed fraud rate is 2.10% in the control group (current model) and 2.02% in the treatment group (new model) — a difference of -0.08 percentage points (the new model had slightly lower fraud). The 95% confidence interval for that difference runs roughly from -0.34 to +0.18 percentage points. Since the interval’s upper bound (+0.18) sits well below the 0.5-percentage-point margin, non-inferiority is concluded: there’s evidence the new model doesn’t push fraud beyond what’s tolerable — and in this case the central estimate even suggests a slight improvement, though the interval doesn’t support that claim with the same confidence as non-inferiority.

11. Simple code example

```
import numpy as np
from statsmodels.stats.proportion import confint_proportions_2indep

margin = 0.5 # percentage points; a business decision, not a statistical one

fraud_control = (252, 12000) # illustrative: (cases, total)
fraud_treatment = (242, 12000) # illustrative

ci_low, ci_high = confint_proportions_2indep(
    fraud_treatment[0], fraud_treatment[1],
    fraud_control[0], fraud_control[1],
    method="wald",
)
ci_low_pp, ci_high_pp = ci_low * 100, ci_high * 100

non_inferior = ci_high_pp < margin
print(f"95% CI: [{ci_low_pp:.2f}, {ci_high_pp:.2f}] pp | margin: {margin} pp")
print(f"Non-inferiority concluded: {non_inferior}")
```

The decision doesn't hinge on a single p-value, but on comparing the confidence interval's upper bound directly against the margin agreed on beforehand.

Heterogeneity testing and novelty-effect decay

1. What problem does this solve?

An experiment shows a positive average effect. But is that effect the same for everyone, or is one subgroup driving the whole result while others feel nothing? And does the effect measured in week one still hold in week eight, or was it just curiosity around something new that faded once the novelty wore off?

These are two different questions that get conflated often: **effect heterogeneity** asks whether the effect differs across groups at a given point in time. **Novelty-effect decay** asks whether the effect changes over time, within the same group. This document covers both, because they tend to show up together when reading an experiment's results.

2. Intuition

An overall average can hide very different stories. An average effect of +3% conversion could mean +12% for new customers and +0% for existing ones — or it could mean +3% spread evenly across everyone. Without testing this formally, both situations look identical in the headline number, but they call for completely different product decisions.

The same logic applies over time: a +5% effect measured only in week one may just reflect genuine curiosity about something new on the screen — a “novelty effect” — that dissolves once the new feature becomes routine.

3. Simple explanation

For heterogeneity, the formal approach is not to eyeball the effect separately for each subgroup (that will always look different just by chance). Instead, a statistical model includes an **interaction term** — something like “treatment $\times$ subgroup” — and tests whether that term is statistically different from zero. That answers: is the difference in effect across subgroups larger than sampling variation alone would produce?

For novelty decay, the effect is estimated separately in successive time windows since launch (week 1, week 2, week 3...) and checked for a downward trend.

4. Easy conceptual example

Imagine a streaming app testing a new home-screen layout. The average effect on daily watch time is positive. Splitting by user type, the effect is large among subscribers with less than a month of tenure and nearly zero among long-time subscribers — plausible, since people with established habits react less to a visual change. Looking week by week, the effect starts high and halves within a month — also plausible, since part of the initial reaction is just curiosity about the new screen.

5. How it works, broadly

Heterogeneity: fit a model with the treatment effect, a subgroup indicator, and the interaction between the two. The interaction coefficient measures how much the effect differs between the subgroup in question and the reference subgroup; its standard error and p-value indicate whether that difference is statistically reliable.

Novelty decay: estimate the effect (and its confidence interval) separately for each time window since launch, and look at the trajectory of those estimates — ideally with a formal trend test rather than only visual inspection.

6. What the result means

A statistically significant interaction term is evidence that the effect genuinely differs across the tested subgroups — not just sampling noise. A consistent decay trajectory, with the effect in the final weeks clearly smaller than in the first, is evidence that part of the effect measured early on was temporary.

7. How to interpret it

Real heterogeneity changes the launch decision: it may make sense to ship only to the subgroup that responded, rather than to everyone. Novelty decay changes the evaluation horizon: deciding based on week one alone can overstate the long-run gain. The two readings are independent of each other — an effect can be homogeneous across subgroups and still decay over time, or heterogeneous and stable.

8. When it is useful

Common when reading product experiments: testing whether the effect of a change varies by loyalty tier, acquisition channel, device type, or another relevant cut, and whether the effect observed right after launch holds up in the following weeks.

9. Important caveats

Testing many subgroups raises the chance of finding a “significant” difference purely by chance — this is the **multiple comparisons** problem. Testing 10 subgroups at the 5% significance level yields, by pure chance, close to half a “significant” subgroup even with no real heterogeneity. So a single heterogeneity finding — especially from a subgroup chosen after looking at the data — deserves far more caution than one defined before the experiment and confirmed consistently.

10. A small numerical example

An A/B test measures an effect of +4.0 percentage points in conversion for the “new customers” group (standard error 1.1) and +0.4 points for the “existing customers” group (standard error 0.9). The difference between the two effects is 3.6 points, with a combined standard error of roughly 1.45 — giving a z-statistic near 2.5, above the usual significance threshold. That is evidence of genuine heterogeneity, not just sampling coincidence.

Separately, the effect measured week by week since launch was +8.5 points in week 1, +5.2 in week 4, and +1.8 in week 8 — a consistent decline consistent with a novelty effect.

11. Simple code example

```
import statsmodels.formula.api as smf

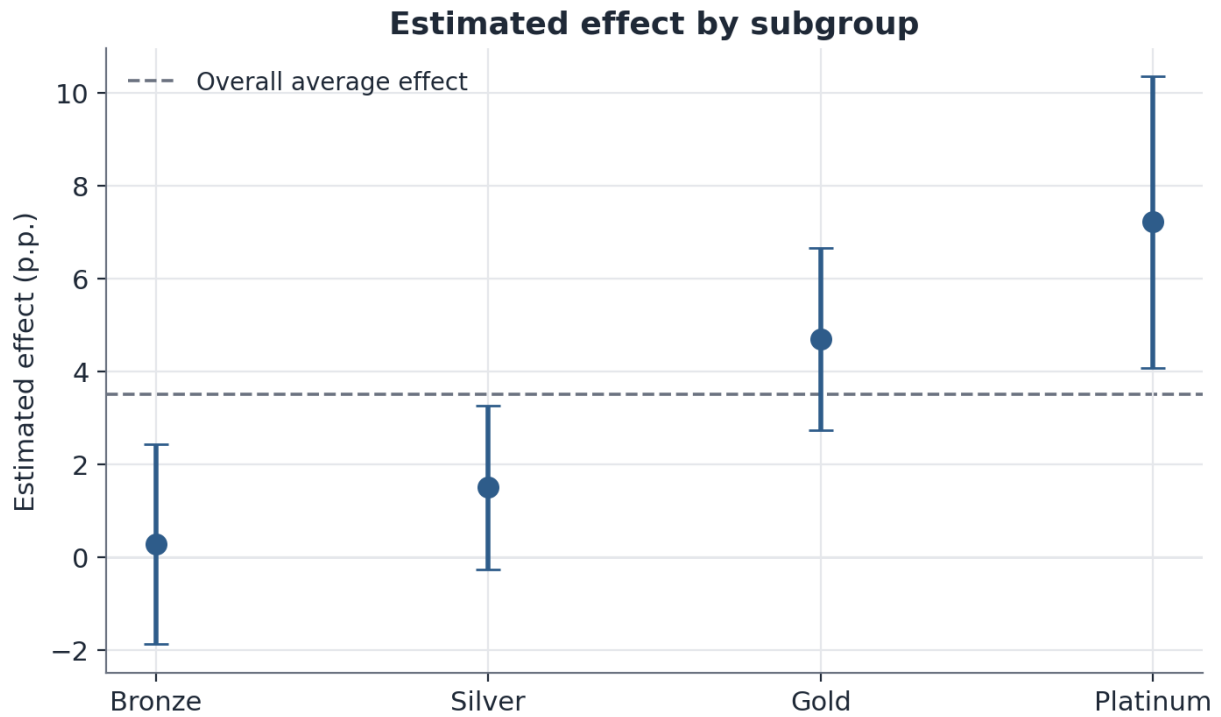
# treatment: 0/1; subgroup: categorical; conversion: continuous metric
model = smf.ols("conversion ~ treatment * subgroup", data=df).fit()
```

```

print(model.summary())
# the coefficient on "treatment:subgroup[T.new]" is the interaction term

# novelty decay: effect by week since launch
weekly_effects = (
    df.groupby("weeks_since_launch")
      .apply(lambda g: smf.ols("conversion ~ treatment", data=g).fit().params["treatment"])
)
print(weekly_effects)

```



Illustrative data, not drawn from any real project.

Figure 13: Estimated effect by subgroup, with confidence intervals and the overall average effect line

Statistical power and minimum detectable effect (MDE)

1. What problem does this solve?

You're designing an experiment — testing a change in an app, a new campaign, a new process — and need to decide on a sample size before starting. The question rarely asked, but essential, is: **given the sample size you can actually run, is there any real chance of detecting the effect you expect, if it truly exists?** Running an expensive experiment for weeks only to end up unable to conclude anything — because the design never had enough power to begin with — is one of the most common and most avoidable wastes in product and growth teams.

2. Intuition

Every statistical test has a limited capacity to see small effects. The smaller the sample and the noisier (more variable) the metric, the blurrier the test's vision — small effects get lost in sampling noise even when they're real. **Statistical power** is the probability that a test detects a true effect of a given size, given the experiment's design. The **minimum detectable effect (MDE)** is the mirror image of the same idea: instead of asking “what's the chance of detecting a 3% effect?”, it asks “what's the smallest effect this design can detect with reasonable confidence (typically 80%)?”

3. Plain-language explanation

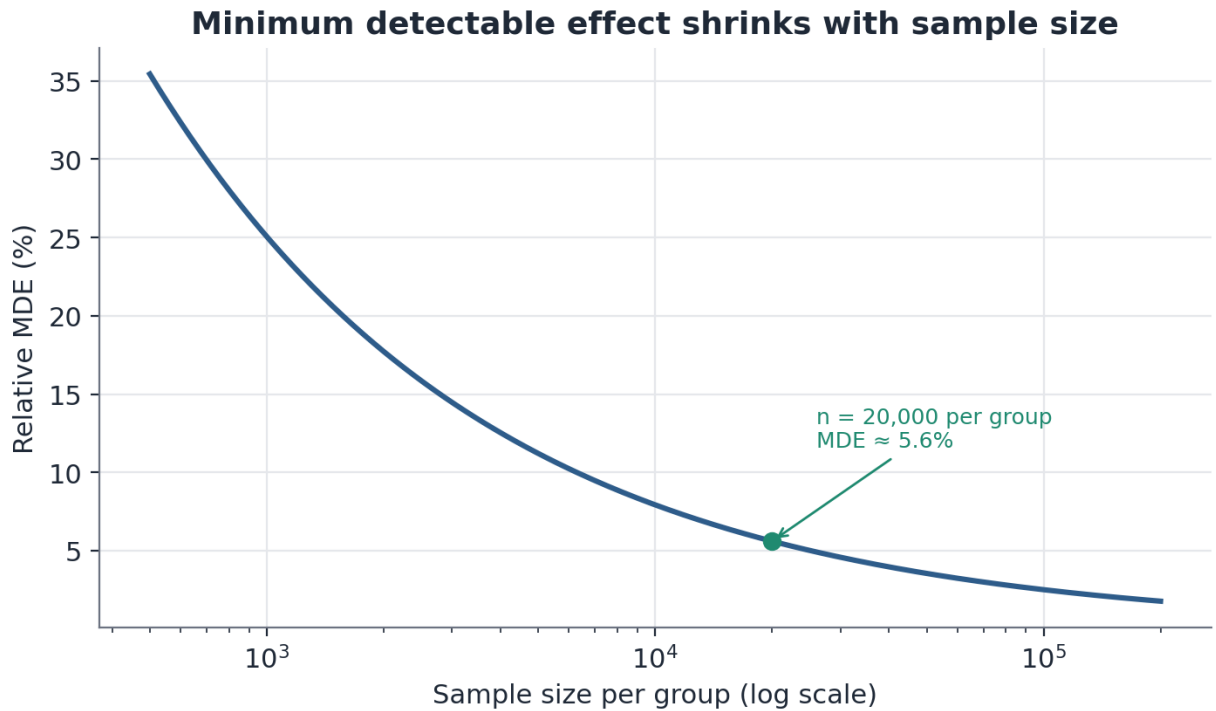
Four ingredients determine an experiment's MDE: sample size, the metric's variance (spread), the accepted significance level (the probability of a false positive, usually 5%), and the desired power (usually 80%). Increasing the sample or reducing the metric's variance lowers the MDE — the test starts seeing progressively smaller effects. But the gain isn't linear: doubling the sample doesn't cut the MDE in half, it divides it by roughly 1.4 (the square root of two). That matters in practice: past a certain point, spending much more experiment time to shrink the MDE from 3% to 2.5% may not be worth the cost.

4. Easy conceptual example

Picture a subscription app testing a new signup flow. The team realistically expects a 2-percentage-point lift in conversion. Before running the experiment, the MDE is calculated for the traffic available over two weeks. If the calculated MDE is 5 percentage points, the experiment simply doesn't have the power to detect a 2-point gain, even if it truly exists — running it as planned would burn time while learning little. The fix is to extend the duration, increase the share of traffic exposed, or accept that only larger effects will be detectable.

5. How it works, broadly

1. Define the primary metric and estimate its variance (from historical data whenever possible).
2. Set the significance level (usually $\alpha = 0.05$) and the target power (usually 80%).
3. Set the available sample size (or, in the reverse direction, the minimum effect you want to be able to detect).
4. Use the relationship between these four quantities — sample size, variance, significance, and power — to solve for the missing one: the MDE (given the sample size) or the required sample size (given a target MDE).



Illustrative data, not drawn from any real project.

Figure 14: Minimum detectable effect shrinking with sample size, showing diminishing returns

5. Compare the resulting MDE against the effect size that would actually matter for the business. If the MDE is larger than any plausible effect, the design needs to change before the experiment runs — not after.

6. What the result means

An MDE of, say, 4% means: “this experiment design can detect, with 80% confidence, a true effect of 4% or larger — smaller effects will likely go unnoticed even if they exist.” It’s not a guarantee that the true effect is 4% — it’s a property of the experiment’s design, computed before any data is collected.

7. How to interpret it

A high MDE isn’t, by itself, a problem — the problem is a high MDE relative to the effect that would plausibly matter. If a tested change would only be worth shipping given an effect of at least 5%, and the calculated MDE is 2%, the design is over-powered (fine for catching even small effects, but possibly more expensive than needed). If the MDE is 8% and the plausible effect is 2%, the design is under-powered — the sample needs to grow, the metric needs to get less noisy, or the team needs to accept that only large effects will be visible.

8. When it’s useful

In planning any controlled experiment, before committing time and traffic to a design that might never have had a chance of showing anything. It’s also useful mid-experiment, when the available

window changes — a business deadline forces an earlier cutoff, or traffic dropped unexpectedly. Recalculating the MDE at that point, using the sample size actually reached, avoids two traps: concluding “no effect” when the experiment never had the power to detect the expected effect in the first place, or continuing to run longer than necessary.

9. Important caveats

- The MDE depends on the estimated variance of the metric — if that estimate is poor (based on little history, or on an atypical period), the calculated MDE will also be off.
- Power and MDE say nothing about how likely the true effect is to be a given size — they only describe what the design is capable of detecting, should the effect exist.
- Recomputing the MDE after peeking at the experiment’s own data (rather than beforehand, or with an independently fixed window) to decide whether it’s “worth continuing” can introduce bias — stopping decisions need a plan set in advance.
- Highly skewed metrics (a few extreme values dominating the variance) inflate the estimated variance and, with it, the MDE — it’s worth checking whether the metric or a transformation of it is more stable.

10. A small worked example

A convenience-store chain wants to test a new window display. The primary metric is the visitor-to-buyer conversion rate, currently at 20%. With a 5% significance level and 80% target power, a sample of 20,000 visitors per group (treatment and control) yields a relative MDE of roughly 8% — meaning the design can reliably detect conversion rising from 20% to about 21.6% or more. If the team realistically expects only a 3% gain, this design isn’t powered enough: it would need a much larger sample, around 140,000 visitors per group, to reach an MDE compatible with an effect of that size.

11. Simple code example

```
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

analysis = NormalIndPower()

# minimum detectable effect (in effect-size units) for a fixed n per group
mde_effect_size = analysis.solve_power(
    effect_size=None, nobs1=20_000, alpha=0.05, power=0.8, ratio=1.0
)

# sample size needed to detect a specific effect
target_effect_size = proportion_effectsize(0.23, 0.20) # 20% -> 23%
n_required = analysis.solve_power(
    effect_size=target_effect_size, nobs1=None, alpha=0.05, power=0.8, ratio=1.0
)

print(f"MDE (effect size) for n=20,000: {mde_effect_size:.4f}")
print(f"Required sample per group for a 20% -> 23% effect: {n_required:.0f}")
```

The same logic works for recalculating power mid-experiment: swap `nobs1` for the sample size actually collected so far and solve for `power` instead of for `nobs1` or `effect_size`.

Spatial spillover and cannibalization analysis

1. What problem does this solve?

You tested a change in a handful of stores, cities, or regions — a new campaign, a new service, a price change — and measured a positive effect by comparing those treated units to control units. But what if part of that “effect” isn’t new value creation at all, just customers who migrated from a nearby control store to the treated one? In that case, the experiment isn’t measuring real gain — it’s measuring displacement, and the nearest control unit no longer serves as a neutral baseline, because it was affected too.

2. Intuition

A geographically spread-out experiment implicitly assumes control units keep behaving “normally,” with no influence at all from what’s happening at the treated units. That assumption breaks when treatment and control compete for the same audience — the same city, the same delivery radius, the same customer base. If a treated store pulls in customers who used to buy at a control store a few kilometers away, that control store will sell less during the experiment — not because nothing changed for it, but because the treatment “stole” part of its traffic. Comparing treatment against that contaminated control inflates the apparent effect.

3. Plain-language explanation

The core diagnostic logic is to compare two kinds of control: control units **near** the treated units (which may be experiencing spillover) and control units **far** from them (outside any plausible radius of direct competition). If the effect estimated using nearby controls is much larger than the effect estimated using distant controls, that gap is a signal of cannibalization: part of the measured “gain” against the nearby control is actually its loss, not net creation somewhere else.

4. Easy conceptual example

Picture a convenience-store chain testing a new loyalty program in a handful of stores in one city. Comparing those stores against control stores in the same neighborhood, the program appears to lift sales by 12%. Comparing the same treated stores against control stores in distant neighborhoods, with no treated stores nearby, the measured lift drops to 6%. The gap between 12% and 6% suggests that much of the “gain” at the nearby stores came from customers who simply switched stores within the same neighborhood — not from new customers or a real increase in spending.

5. How it works, broadly

1. Map the location of every experimental unit (treated and control) and compute the distance from each control to the nearest treated unit.
2. Define distance bands — for example, controls within 3 km (“nearby,” possibly contaminated) and controls beyond 10 km (“distant,” unlikely to face direct competition).
3. Estimate the treatment effect separately against each control group.
4. Compare the magnitude of the two effects. A large gap between them is evidence of spillover; similar effects suggest that spatial contamination, if it exists, is small.
5. When possible, also examine how the effect on control units varies continuously with distance to the nearest treated unit — genuine cannibalization tends to fade smoothly with distance rather than drop off abruptly.

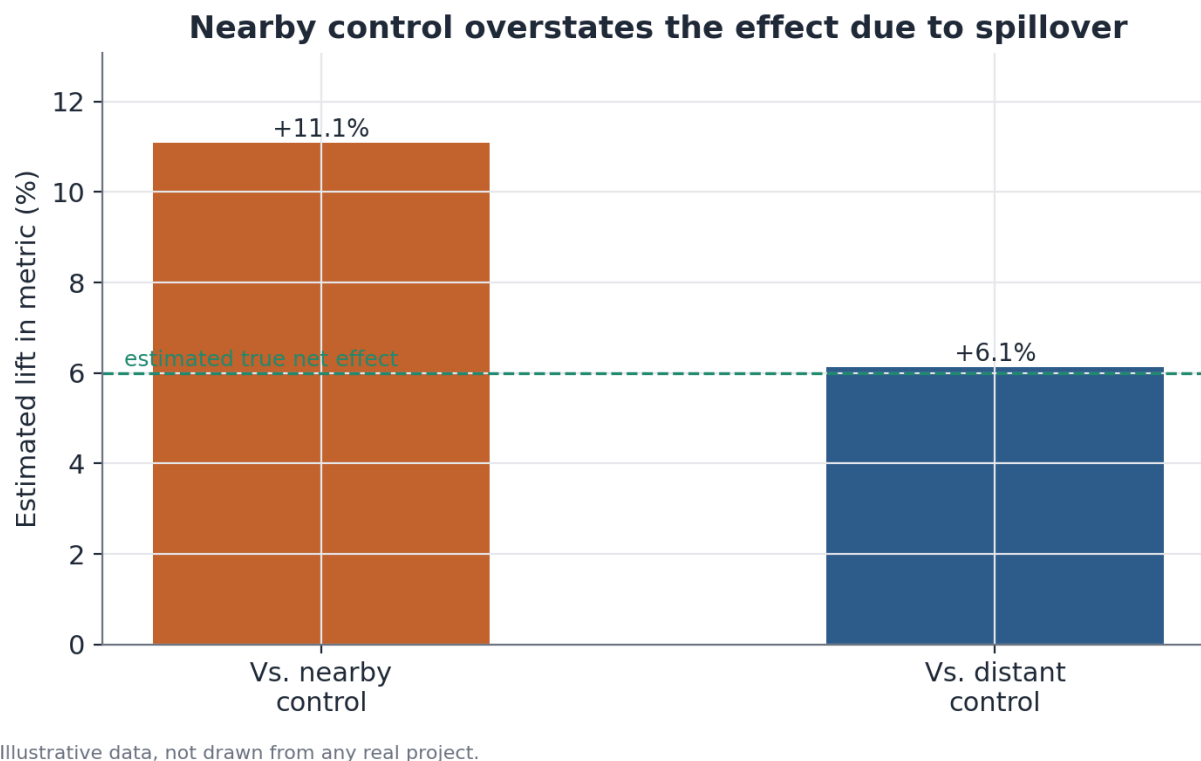


Figure 15: Estimated effect using nearby control versus distant control

6. What the result means

If the effect measured against nearby controls is consistently larger than against distant controls, the “true net effect” — the one that reflects real value creation, not just reallocation — sits closer to the distant-control estimate. The nearby-control effect measures something different: how much the treated unit gained at the immediate neighborhood’s expense, plus whatever it created that’s genuinely new.

7. How to interpret it

Not all cannibalization is a problem — sometimes it’s exactly expected and acceptable (a brand may know a new product will steal sales from an older one in the same line, and that’s part of the plan). What matters is not confusing displacement with net creation when deciding whether it’s worth rolling the change out to every unit. If the goal is market growth, the net effect (via the distant control) is the relevant metric. If the goal is just understanding the treated unit’s relative performance against its immediate neighborhood, the nearby-control effect also has value — but for a different question.

8. When it’s useful

In any experiment with geographically close units competing for the same audience — physical stores, delivery zones, service coverage regions. It’s especially important when a positive experiment result looks “too good” and is about to justify a large-scale rollout decision, where undetected cannibalization inflates expected gains and can lead to overly optimistic revenue projections.

9. Important caveats

- The cutoff between “nearby” and “distant” is a design choice that needs justification — grounded in a real competition radius (for example, the distance customers typically travel), not an arbitrary number.
- “Distant” controls need to genuinely sit outside any treatment influence — if the true competition radius is larger than assumed, even the “distant” controls may be partially contaminated, understating the gap between the two groups.
- A small or null net effect (via distant control) alongside a large effect via nearby control doesn’t mean the change “didn’t work” — it means it redistributed demand rather than creating new demand, which can still be a valuable outcome depending on the business goal.
- Small numbers of treated and control units (few stores, few regions) make the effect estimates unstable; in that setting, the uncertainty around the comparison between the two effects needs to be reported, not just the point estimate.

10. A small worked example

A store chain tests extended opening hours in 15 stores in a metro area. Twenty control stores in the same neighborhoods sit within 3 km of some treated store; another 25 control stores, used as the distant comparison group, sit more than 12 km from any treated store.

- Average daily sales, treated stores: \$8,400 (before) → \$9,660 (after).
- Average daily sales, nearby control: \$8,100 (before) → \$7,930 (after) — a 2.1% drop.
- Average daily sales, distant control: \$8,050 (before) → \$8,170 (after) — a 1.5% rise.

The effect estimated against the nearby control is roughly 17.3% (because the control fell while the treatment rose). The effect estimated against the distant control is roughly 13.5%. The gap of about 3.8 percentage points is consistent with some degree of cannibalization from neighboring stores — part of the increase at treated stores came from customers who used to buy at the nearby control stores, not purely from more overall spending in the region.

11. Simple code example

```
import numpy as np
import pandas as pd

# illustrative data: one row per store, with distance to the nearest treated store
stores = pd.DataFrame({
    "store_id": range(1, 61),
    "treated": [True] * 15 + [False] * 45,
    "distance_km": [0] * 15 + list(np.random.default_rng(1).uniform(0.5, 20, 45)),
    "sales_before": np.random.default_rng(2).normal(8100, 400, 60),
    "sales_after": np.random.default_rng(3).normal(8300, 500, 60),
})

nearby_control = stores[(~stores["treated"]) & (stores["distance_km"] < 3)]
distant_control = stores[(~stores["treated"]) & (stores["distance_km"] > 10)]
treated = stores[stores["treated"]]

def pct_change(df):
```

```

    return (df["sales_after"].mean() / df["sales_before"].mean() - 1) * 100

change_treated = pct_change(treated)
change_nearby = pct_change(nearby_control)
change_distant = pct_change(distant_control)

effect_vs_nearby = change_treated - change_nearby
effect_vs_distant = change_treated - change_distant

print(f"Effect vs. nearby control: {effect_vs_nearby:.1f} pp")
print(f"Effect vs. distant control: {effect_vs_distant:.1f} pp")
print(f"Gap (sign of cannibalization): {effect_vs_nearby - effect_vs_distant:.1f} pp")

```

Cluster-robust standard errors

1. What problem does this solve?

The standard way of computing a standard error assumes every observation in the sample carries new, independent information. But what happens when many observations come from the same group — the same store, the same classroom, the same region — and resemble each other for reasons that have nothing to do with the variable you are testing? Treating those observations as independent makes the standard error look smaller than it really is, and can lead to calling something “significant” when it isn’t.

2. Intuition

Picture measuring customer satisfaction across 20 stores, with 50 responses per store — 1,000 observations in total. If each store has its own service culture, manager, location, and clientele, responses from the same store tend to look alike because of those shared factors, not just coincidence. In practice, you don’t have 1,000 independent pieces of information — you have something closer to 20 “blocks” of information, each made of correlated responses. Computing the standard error as if these were 1,000 independent observations overstates how precise the estimate really is.

3. Simple explanation

Each new observation within the same cluster carries less additional information than an observation from a different cluster, because part of what it shows was already predictable from the other observations in that same group. The cluster-robust standard error recognizes this internal correlation and adjusts the calculation to reflect the effective number of “independent units of information” — which sits much closer to the number of clusters than to the total number of observations.

4. Easy conceptual example

A gym chain tests a new training program in 8 locations (treatment) against another 8 locations (control), with about 40 members per location. Members at the same location share the same instructor, the same peak hours, the same neighborhood — factors that affect the outcome regardless of the program being tested. A standard error that ignores this treats the 640 members as 640 independent sources of evidence; in practice, the real evidence is much closer to coming from 16 locations.

5. How it works, broadly

Instead of assuming model errors are independent across all observations, the cluster-robust standard error allows errors to be correlated within each cluster (without requiring a specific form for that correlation) and only assumes independence **between** different clusters. The variance-covariance matrix of the coefficients is recomputed by summing contributions cluster by cluster, rather than observation by observation — which typically produces a larger (more conservative, more honest) standard error than the naive calculation.

6. What the result means

The point estimate of the effect does not change — only the standard error around it. A confidence interval computed with cluster-robust standard errors is wider, reflecting more faithfully the real

uncertainty of the estimate, given that the effective amount of independent information is smaller than the raw row count in the dataset.

7. How to interpret it

If a result was “significant” under the naive standard error and stops being significant under the cluster-robust one, that is not a flaw of the method — it is the correction showing that the original confidence was artificially inflated. The stronger the within-cluster correlation and the more uneven the cluster sizes, the larger this gap tends to be.

8. When it is useful

Whenever the unit of randomization or data collection differs from the unit of analysis — for example, an experiment randomized by store but analyzed at the customer level, a household survey where several people in the same household respond, or data organized by geographic region across multiple time periods.

9. Important caveats

With **few clusters** (a common rule of thumb: fewer than 30-40), even the cluster-robust standard error can be unstable and biased downward — the asymptotic correction it relies on needs enough clusters to work well. In that setting, alternative methods such as randomization inference or wild-cluster bootstrap are usually preferable (see the corresponding documents).

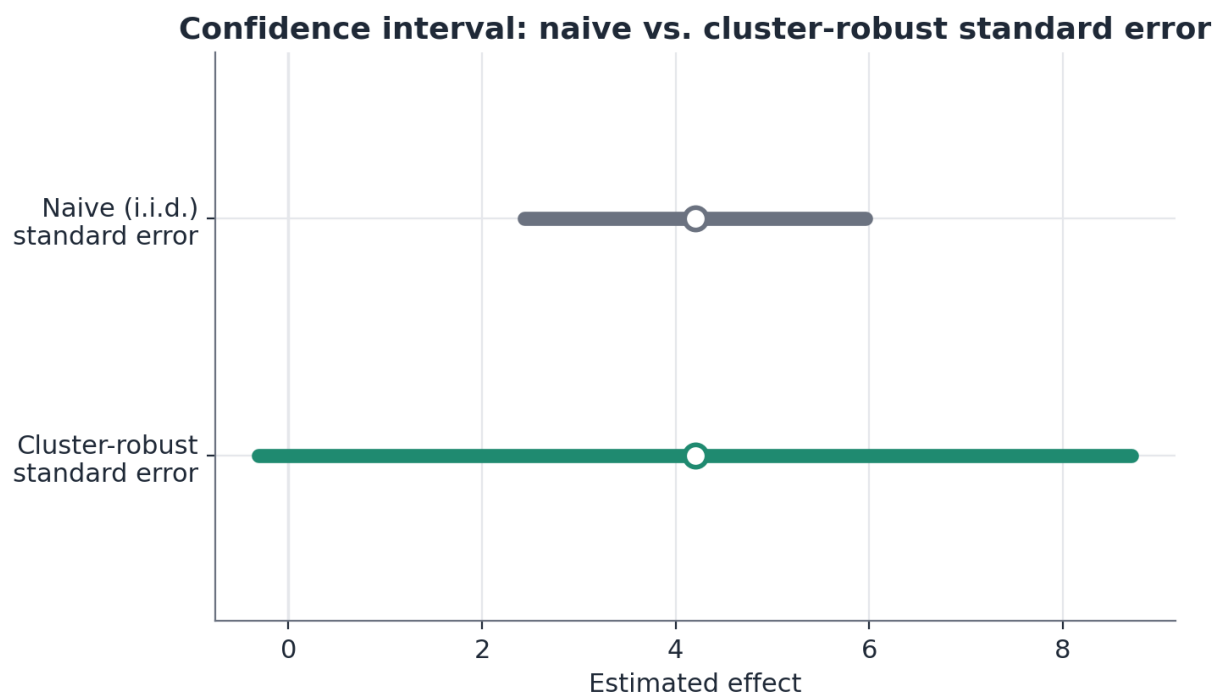
10. A small numerical example

In a simple model with 1,000 observations across 20 clusters, suppose the estimated coefficient of interest is 5.0. The naive standard error (treating the 1,000 observations as independent) is 0.45, giving a t-statistic 11.1 — strongly significant. Recomputing it as a cluster-robust standard error, accounting for the correlation within each of the 20 stores, the standard error rises to 1.60, giving $t = 3.1$ — still significant, but with a much more realistic confidence margin.

11. Simple code example

```
import statsmodels.formula.api as smf

model = smf.ols("satisfaction ~ treatment", data=df).fit(
    cov_type="cluster",
    cov_kwds={"groups": df["store_id"]},
)
print(model.summary())
```



Illustrative data, not drawn from any real project.

Figure 16: Confidence interval computed with the naive standard error compared to the cluster-robust standard error

Randomization inference (permutation test)

1. What problem does this solve?

Common statistical tests (the t-test, for example) rely on approximations that work well when there are many observations. But what happens when an experiment has few units — few stores, few cities, few classrooms — because randomization happened at an aggregate level, not at the level of each person? With few units, those approximations stop being reliable. Randomization inference solves this without relying on any approximation at all: it uses the actual randomness of the experiment’s design to build the test.

2. Intuition

If treatment was assigned by a lottery, there is a finite, known number of ways that lottery could have turned out. The method’s central question is: “among all the possible ways of drawing who would be treated, what fraction of them would produce an effect as large as (or larger than) the one I actually observed?” If the answer is “almost none,” the observed result is unusual enough to be evidence of a real effect, not merely of the particular draw that happened to occur.

3. Simple explanation

The method reshuffles, across the available units, every possible combination (or a large sample of them) of who would be “treatment” and who would be “control,” recomputing the statistic of interest (usually the mean difference) for each reshuffle. This generates an entire distribution of possible outcomes under the hypothesis that treatment had no effect at all. The p-value is the fraction of that distribution as extreme as the actually observed result.

4. Easy conceptual example

Consider an experiment with only 4 stores, of which 2 will be drawn to receive a new window display and 2 will stay as they are. There are exactly $\binom{4}{2} = 6$ different ways to choose which 2 stores receive the treatment. Each of those 6 ways produces a different sales difference between “treatment” and “control.” The test’s reference distribution is made up of exactly those 6 possible differences — nothing more.

5. How it works, broadly

1. Compute the observed statistic (for example, the mean difference between the treated group and the control group, under the experiment’s actual assignment).
2. Enumerate every possible treatment/control reassignment among the same units (or, when the number of combinations is too large to enumerate, draw a large sample of reassignments, typically thousands).
3. For each reassignment, recompute the same statistic, keeping each unit’s observed value fixed — only the treatment/control label changes.
4. The p-value is the proportion of those recomputed statistics that is as extreme as, or more extreme than, the statistic observed under the actual assignment.

6. What the result means

A low p-value says the experiment’s actual assignment produced an unusual result, compared to every other way the lottery could have turned out. That is evidence the treatment had an effect — without relying on any assumption about the shape of the data’s distribution.

7. How to interpret it

The method’s validity depends entirely on treatment assignment having genuinely been random (or on the design allowing reassignments to be treated as equally likely). Unlike tests that assume normality or a large sample, here the only assumption needed is the genuine randomness of the assignment — which makes the method especially trustworthy when that randomness is guaranteed, and especially unsuitable when it is not.

8. When it is useful

Particularly useful when the number of experimental units is small — few geographic zones, few stores, few classrooms — a common situation in experiments randomized at the cluster level (geography, store, organizational unit), where traditional asymptotic tests lose reliability.

9. Important caveats

With few units, the number of possible reassignments is also small, which limits how fine-grained the p-value can be — with 4 stores split 2-and-2, for example, the smallest possible p-value is $1/6 \approx 0.167$, never lower than that, no matter how large the effect. This is a structural limit of the method with very small samples, not an implementation flaw.

10. A small numerical example

Six stores take part in an experiment, three drawn for treatment and three for control. Observed weekly sales are: 12, 15, 9, 22, 18, 11 (in thousands of dollars), with the first three being the treatment group under the actual assignment. The observed mean difference is $(12 + 15 + 9)/3 - (22 + 18 + 11)/3 = 12.0 - 17.0 = -5.0$.

There are $\binom{6}{3} = 20$ possible ways to choose which 3 stores would be treatment. Computing the mean difference for each of the 20 combinations gives the full reference distribution. In this example, 6 of the 20 combinations produce a difference as extreme as (or more extreme than) -5.0 in absolute value — giving a two-sided p-value of $6/20 = 0.30$. That is not a significant result: among the 20 possible ways the treatment draw could have gone, a difference this large is not particularly unusual.

11. Simple code example

```
import numpy as np
from itertools import combinations

sales = np.array([12.0, 15.0, 9.0, 22.0, 18.0, 11.0])
actual_treatment = {0, 1, 2} # indices of the 3 stores actually treated

def mean_diff(treat_idx, control_idx):
```

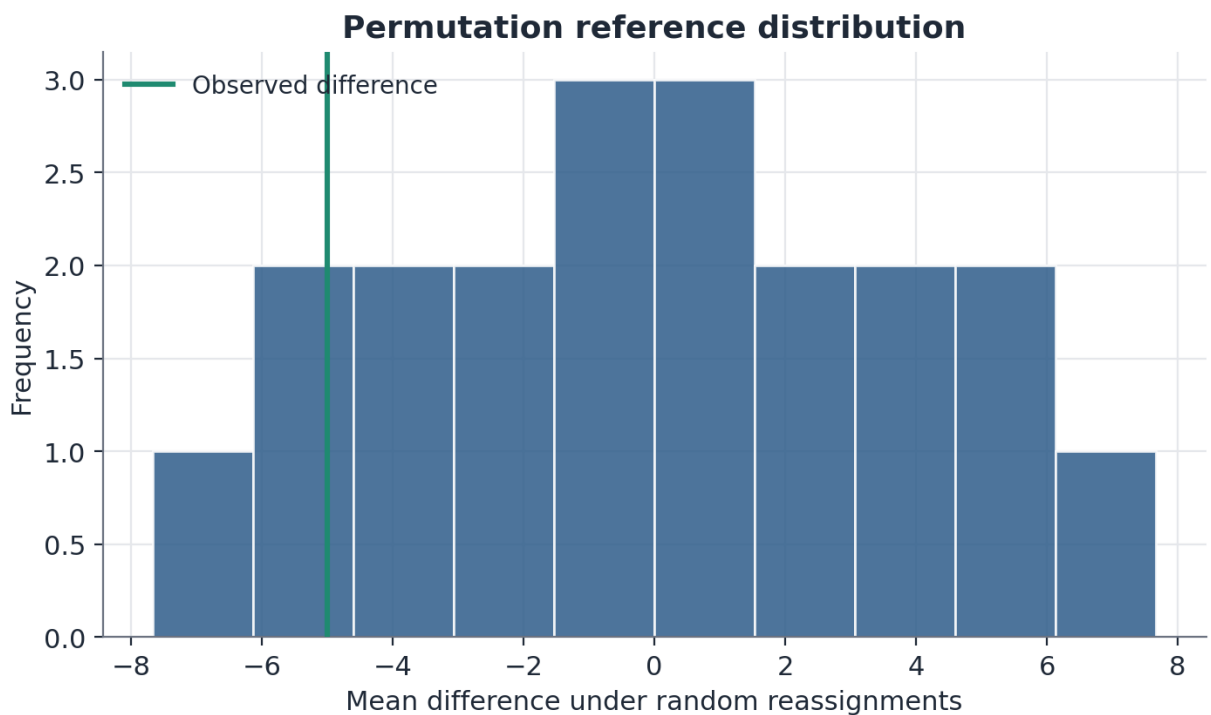
```

    return sales[list(treat_idx)].mean() - sales[list(control_idx)].mean()

all_units = set(range(6))
diffs = []
for combo in combinations(range(6), 3):
    combo = set(combo)
    diffs.append(mean_diff(combo, all_units - combo))

diffs = np.array(diffs)
observed = mean_diff(actual_treatment, all_units - actual_treatment)
p_value = np.mean(np.abs(diffs) >= abs(observed))
print(p_value)

```



Illustrative data, not drawn from any real project.

Figure 17: Permutation reference distribution, showing all 20 possible reassignments and the observed difference

Wild-cluster bootstrap

1. What problem does this solve?

Cluster-robust standard errors work well when there are many clusters, but become unreliable when there are few — a dozen or fewer, for example. In that setting, a hypothesis test based on that approximation can reject the null hypothesis far more often than it should, even when it is true. The wild-cluster bootstrap is a way of building a more reliable reference distribution in that situation, without relying on the asymptotic approximation that fails precisely when clusters are few.

2. Intuition

Instead of trusting a theoretical formula for the standard error, the method generates many alternative, plausible versions of the data — preserving the real dependence pattern within each cluster — and observes how the test statistic varies across those versions. That observed variation becomes the ruler for judging whether the real result is unusual.

3. Simple explanation

The wild-cluster bootstrap does not resample individual data rows, nor does it draw entire clusters with replacement (as an ordinary cluster bootstrap does). Instead, it takes the **residuals** from an already-fitted model, multiplies **every residual belonging to the same cluster by the same random number** (typically $+1$ or -1 , drawn cluster by cluster), adds those perturbed residuals back onto a prediction under the null hypothesis, and refits the model. Repeating this thousands of times produces an entire distribution of test statistics: “this is what could have happened, under the null hypothesis.”

4. Easy conceptual example

Imagine a chain of 6 hospitals testing a new triage protocol, with only 6 clusters (hospitals) in total — too few to trust a conventional cluster-robust standard error. The wild-cluster bootstrap generates thousands of alternative scenarios in which the deviations (residuals) observed at each hospital are “flipped” or “kept” as a block, randomly, hospital by hospital — preserving the pattern that, within a given hospital, deviations tend to move together.

5. How it works, broadly

1. Fit the model under the null hypothesis (for example, treatment effect equal to zero) and obtain the residuals from that restricted fit.
2. For each bootstrap replicate, draw a random sign ($+1$ or -1) **per cluster** — every observation in the same cluster gets the same sign in that replicate.
3. Multiply each cluster’s residuals by the sign drawn for that cluster, creating a synthetic dependent variable.
4. Refit the model to that synthetic variable and recompute the test statistic.
5. Repeating steps 2 through 4 many times (typically 999 or more replicates) builds the test statistic’s reference distribution under the null hypothesis.

6. What the result means

The wild-cluster bootstrap p-value is the fraction of replicates in which the recomputed statistic is as extreme as, or more extreme than, the statistic observed in the real data. Unlike the conventional cluster-robust standard error, this reference distribution is built directly from the data, without relying on a normal approximation that only works well with many clusters.

7. How to interpret it

A low p-value from the wild-cluster bootstrap is more trustworthy evidence, with few clusters, than an equally low p-value from a conventional cluster-robust standard error — because the method does not depend on the asymptotic assumption that fails precisely in that setting.

8. When it is useful

Especially indicated when there are few clusters (a common range in the applied literature is below 30, with extra caution below 10-15), and the effect of interest comes from a regression model — with or without control variables — rather than a simple comparison of means between two groups (in that simpler case, randomization inference is also a direct alternative).

9. Important caveats

The method still depends on the model being reasonably well specified — it solves the standard-error problem under few clusters, but it does not fix a misspecified model. With extremely few clusters (2 or 3, for example), even the wild-cluster bootstrap runs into limitations, since the number of possible sign combinations per cluster also becomes small.

10. A small numerical example

A regression model with 6 clusters produces an estimated treatment coefficient of 4.2, with a conventional cluster-robust standard error of 1.3, giving $t = 3.2$ — apparently significant under the usual asymptotic t reference. Running the wild-cluster bootstrap with 999 replicates (random signs drawn per cluster, 6 sign draws combining $2^6 = 64$ possible patterns), the observed t -statistic of 3.2 lands at the 91st percentile of the bootstrap distribution — corresponding to a two-sided p-value of roughly 0.09, higher (more conservative) than the original asymptotic p-value of about 0.003. The conclusion changes from “strongly significant” to “suggestive, but not conclusive at 5%.”

11. Simple code example

```
import numpy as np
import statsmodels.formula.api as smf

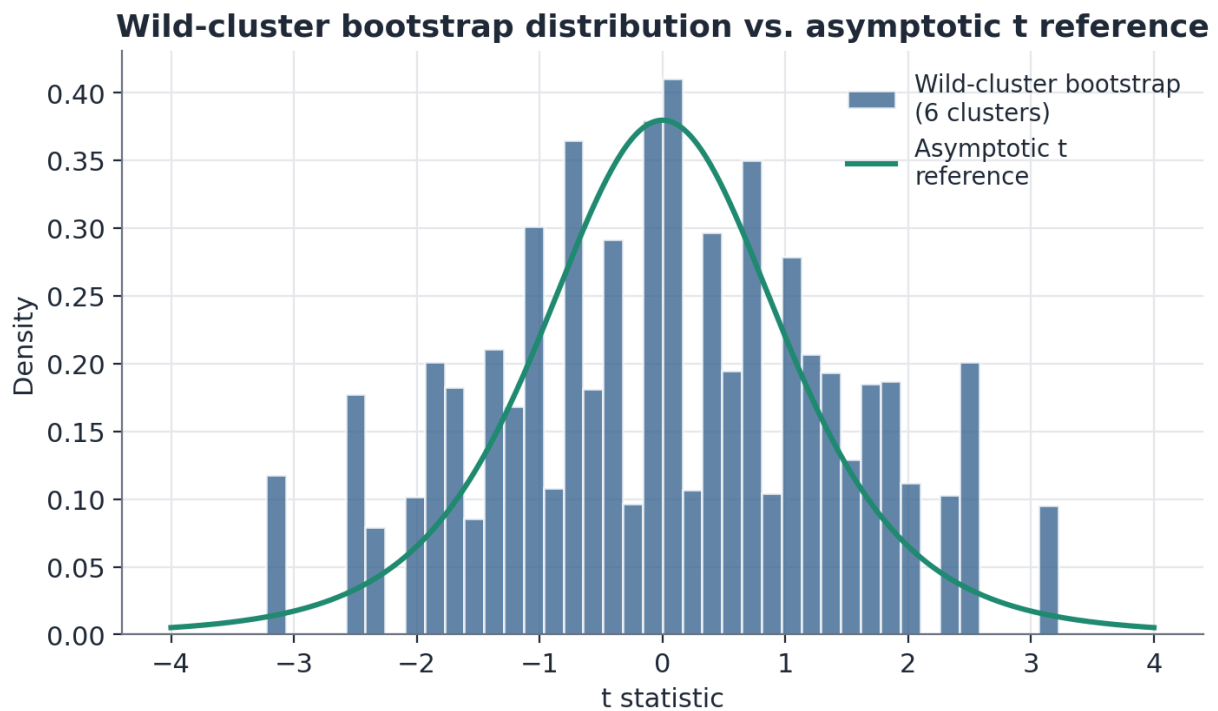
def wild_cluster_bootstrap(df, formula, cluster_col, n_boot=999, seed=0):
    rng = np.random.default_rng(seed)
    restricted_model = smf.ols(formula, data=df).fit()
    residuals = restricted_model.resid
    fitted = restricted_model.fittedvalues
    clusters = df[cluster_col].unique()

    t_boot = []
```

```

for _ in range(n_boot):
    signs = {c: rng.choice([-1, 1]) for c in clusters}
    weight = df[cluster_col].map(signs).values
    y_synthetic = fitted + residuals * weight
    df_boot = df.assign(y_synthetic=y_synthetic)
    boot_model = smf.ols(formula.replace(formula.split("~")[0].strip(), "y_synthetic"),
                        data=df_boot).fit(cov_type="cluster",
                                         cov_kwds={"groups": df_boot[cluster_col]})
    t_boot.append(boot_model.tvalues["treatment"])
return np.array(t_boot)

```



Illustrative data, not drawn from any real project.

Figure 18: Wild-cluster bootstrap distribution compared to the asymptotic t reference, with 6 clusters

Cohort and retention decomposition

1. What problem does this solve?

A product's overall retention rate dropped this quarter. It's tempting to look for a single cause — a price change, a technical issue, a new competitor — but that aggregate drop can come from very different places: maybe customers who joined recently are churning faster than usual, maybe every customer, old and new alike, is leaving a bit more, or maybe no individual is behaving any differently at all, and the drop is purely a composition effect — more new customers joining, and new customers naturally retain worse than older ones in their first few months. Each of these causes calls for a completely different response.

2. Intuition

A single “retention rate,” tracked month over month, blends together people who joined at very different times — and each of those people sits at a different point in their own lifecycle as a customer. A **cohort** is the group of customers who joined during the same period (say, everyone who subscribed in January). Instead of looking at one aggregate retention curve, you look at one curve per cohort, aligned by time since joining (month 0, month 1, month 2...) rather than by calendar date. That separates two things the aggregate metric mixes together: how a specific cohort behaves over its own lifetime, and how the mix of recent versus older cohorts in the total customer base shifts in a given month.

3. Plain-language explanation

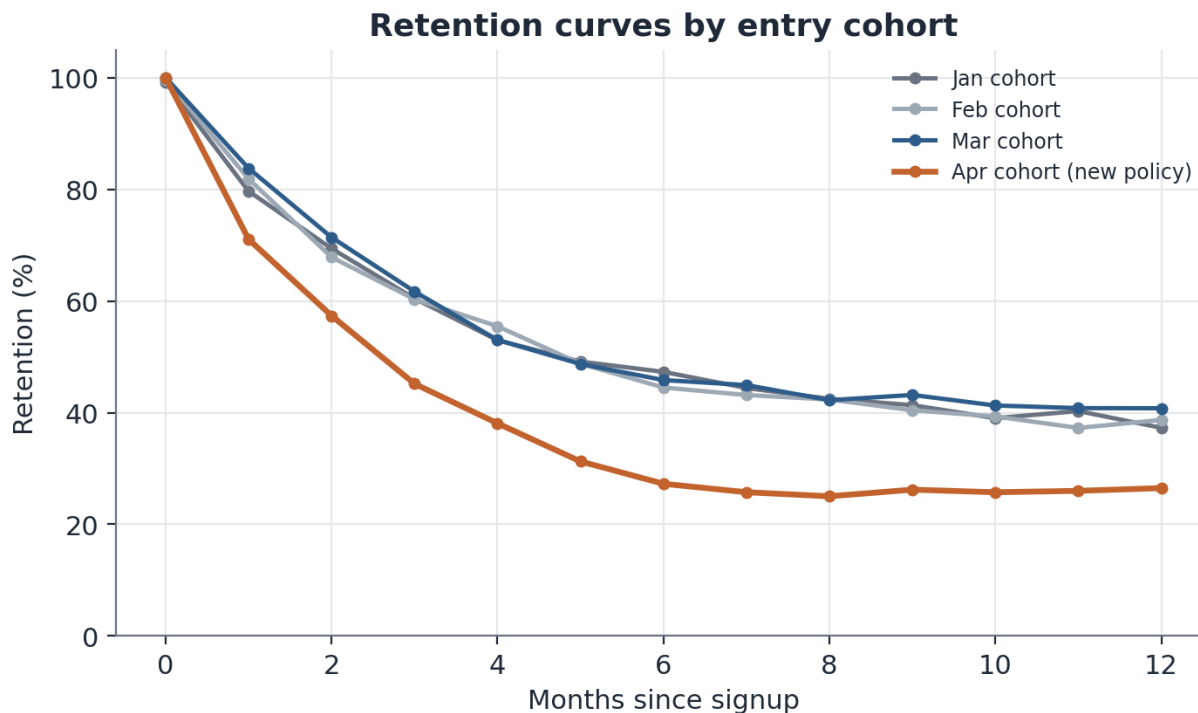
A retention curve is built for each entry cohort: the share of the original cohort still active in each subsequent month since joining. Different cohorts can have similar curves (a sign that customer behavior is stable over time) or different ones (a sign that something changed for whoever joined during a specific period — a product change, an acquisition campaign that brought in a less engaged audience, a technical issue that disproportionately hit people mid-onboarding). Comparing these curves side by side makes it possible to tell whether an aggregate drop comes from one specific cohort, from every cohort equally, or from a shift in the base's composition.

4. Easy conceptual example

Picture a subscription app that notices a drop in average month-3 retention (the fraction of subscribers still active three months after subscribing). Looking at aggregate retention alone, it seems like “customers are leaving more.” But breaking it down by entry cohort makes it clear that the January, February, and March cohorts have curves nearly identical to prior months — only the April cohort, which coincides with a change in the billing process, shows a visible drop in month-3 retention. The cause isn't general; it's specific to one cohort and one identifiable event.

5. How it works, broadly

1. Group customers by the date (month, week) they joined — that's the cohort.
2. For each cohort, compute the share still active in each period since joining (month 0 = 100%, month 1 = retained fraction, and so on).
3. Plot the curves for all relevant cohorts, aligned by time since joining, not calendar date.
4. Compare visually and numerically: are recent cohorts worse than older ones at the same point in the curve? If so, something specific to recent cohorts is happening.



Illustrative data, not drawn from any real project.

Figure 19: Retention curves by entry cohort, with a recent cohort highlighted

5. Separate out the composition effect: even with no individual cohort getting worse, if the current base has a larger share of recent cohorts (which sit at a lifecycle stage with naturally lower retention), the aggregate metric drops purely because of that mix shift.

6. What the result means

If a specific cohort has visibly worse retention than earlier cohorts at the same point in time since joining, that points to a localized cause — something that happened specifically to whoever joined during that period. If every cohort, including older ones, shows a drop starting from a certain calendar month, that points to an event that hit everyone at once, regardless of when they joined. If each cohort's individual curve is stable, yet the aggregate metric still dropped, the explanation lies in composition — the base's mix changed, not its behavior.

7. How to interpret it

It's important not to jump straight to “cohort X got worse, so something broke specifically for them” without checking whether the gap is large enough to not just be sampling noise — smaller cohorts (say, a month with few sign-ups) naturally produce more unstable curves. It's also worth checking whether the cohort's composition changed (for example, a different acquisition channel bringing in a different audience), since that can explain the gap without anything having “broken” in the product — just the incoming audience shifted.

8. When it's useful

Whenever an aggregate retention, churn, or tenure metric moves unexpectedly and the cause isn't obvious. It's especially useful for telling apart a product or operations problem (which typically shows up as a localized drop in one specific cohort, coinciding with a known change) from a structural composition effect (which doesn't call for a point fix, but rather an understanding of why the customer mix is shifting).

9. Important caveats

- Very small cohorts produce unstable curves — be careful comparing cohorts of very different sizes without accounting for that.
- Retention measured in fixed calendar windows (not aligned to time since joining) mixes cohorts at different lifecycle stages and hides the very phenomenon cohort decomposition is meant to reveal.
- A “worse” cohort can reflect a shift in acquisition source, not a product problem — it's worth cross-referencing acquisition-channel data before concluding on a cause.
- Comparisons between cohorts are observational, not experimental — even when a specific cohort worsens right after a known change, the timing coincidence suggests, but doesn't prove, causation; other factors that also shifted during that period could be behind the difference.

10. A small worked example

A hypothetical streaming service tracks month-3 retention (the share of subscribers still active three months after subscribing) for four monthly entry cohorts:

Cohort	Month-3 retention
January	39%
February	38%
March	40%
April	26%

Aggregate retention across the whole base, looked at for the current month alone, dropped from 38% to 34% versus the prior quarter. Breaking it down by cohort makes it clear that January, February, and March sit within the historical range (38–40%) — only the April cohort, 12 to 14 percentage points below the others, is dragging the aggregate average down. The investigation then focuses on what was different for whoever subscribed in April — in this hypothetical case, a change to the card-billing process that coincided exactly with that month.

11. Simple code example

```
import pandas as pd
import numpy as np

# illustrative data: one row per subscriber, entry cohort and month-3 status
rng = np.random.default_rng(5)

cohorts = ["2026-01", "2026-02", "2026-03", "2026-04"]
```

```

base_retention = {"2026-01": 0.39, "2026-02": 0.38, "2026-03": 0.40, "2026-04": 0.26}

records = []
for cohort in cohorts:
    n = rng.integers(800, 1200)
    active_month3 = rng.binomial(n, base_retention[cohort])
    records.append({"cohort": cohort, "subscribers": n, "active_month3": active_month3})

summary = pd.DataFrame(records)
summary["retention_month3"] = summary["active_month3"] / summary["subscribers"]

historical_avg = summary.loc[summary["cohort"] != "2026-04", "retention_month3"].mean()
print(summary[["cohort", "retention_month3"]])
print(f"\nHistorical average (excluding April): {historical_avg:.1%}")
print(f"April cohort retention: {summary.loc[summary.cohort == '2026-04', 'retention_month3'].mean():.1%}")

```

Growth decomposition (total, comparable, and price-volume-mix)

1. What problem does this solve?

A retail chain closes the quarter with revenue up 12% year over year. Leadership is pleased — until someone asks: “did that growth come from selling more to the same customers, or simply from the eight new stores we opened?” And within what already existed: “did we sell more units, or did we just raise prices?” Without separating these pieces, one aggregate growth number can hide either a healthy business or one that’s masking falling volume with price increases.

2. Intuition

Total growth is a sum of fundamentally different things: new stores that didn’t exist in the prior period, and existing stores that were there in both periods. Lumping them together is like comparing the weight gain of someone who started lifting weights with that of a family that just had a baby — the second number grows for a completely different reason than the first.

3. Plain explanation

The method splits growth into two sequential questions:

Question 1 — where did the growth come from? Split total revenue into “comparable” units (present in both periods, also called *like-for-like*) and new units (openings, geographic expansion). Only comparable growth tells you whether the pre-existing business is actually doing better.

Question 2 — within comparable growth, what changed? Decompose it into three effects: **price** (the same items got more or less expensive), **volume** (more or fewer units sold), and **mix** (what customers bought shifted — for example, more high-ticket items in the basket).

4. Simple conceptual example

Picture a coffee shop that sold one type of coffee at \$5. The next month it sells the exact same number of cups, but now charges \$5.50. Revenue went up purely from a **price** effect — no change in volume or mix. Now suppose instead it kept the price the same but sold more cups: that’s a **volume** effect. And if it started selling more of a pricier combo (coffee + pastry) instead of coffee alone, that’s a **mix** effect — the composition of sales shifted even though price and per-unit volume didn’t move on their own.

5. How it works, broadly

1. Classify each unit (store, region, channel) as “comparable” — present in both periods — or “new.”
2. Compute total growth as the revenue difference between the two periods.
3. Isolate the share coming from new units (revenue from units that only exist in the most recent period).
4. What remains is comparable growth — only among units present in both periods.
5. Within comparable growth, decompose the revenue change into a price effect (holding mix and unit volume fixed, varying only average price), a volume effect (holding price and mix fixed, varying quantity), and a mix effect (the residual, capturing the shift in what’s being sold).

Each component answers “how much growth would there have been if only this factor had changed, holding the others constant?” — which is why the three effects sum exactly to total comparable growth.

6. What the result means

A positive total growth number can conceal a comparable base that’s flat or even shrinking, offset by geographic expansion. And positive comparable growth might come mostly from price (raising the question: how much longer can price increases be passed through before volume suffers?) or from real volume (a more robust signal of demand).

7. How to interpret it

Rising volume is usually the healthiest signal — it points to genuine demand growth. Price rising on its own, without volume keeping pace, may reflect passed-through inflation or pricing power, but it isn’t by itself evidence of more customers or more consumption. A positive mix effect might reflect successful upselling, or simply a shift in market demand unrelated to internal strategy — worth investigating the cause before celebrating.

8. When it’s useful

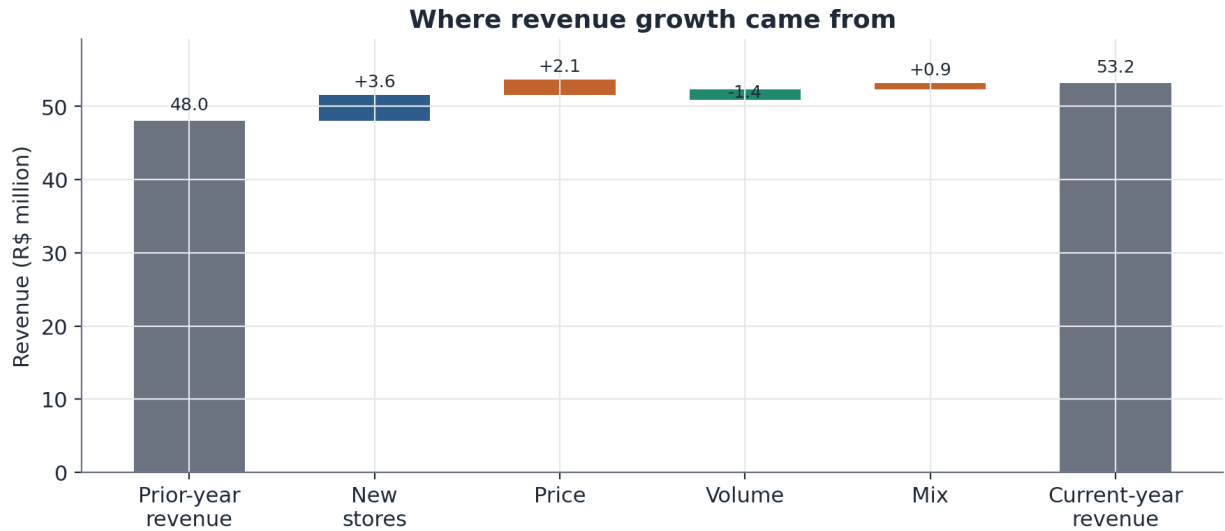
Whenever a reported growth figure blends structurally different sources: expansion versus the existing business, price versus quantity. This is the logic behind “comparable sales” (*same-store sales, like-for-like*) reporting used in retail, franchise networks, and product-portfolio reviews — this exact kind of decomposition is commonly used in that context to separate how much of additional revenue came from opening new locations versus selling more at existing ones.

9. Important caveats

- The price-volume-mix split depends on the order in which effects are computed when price and volume move together — different calculation conventions produce slightly different numbers for each individual component, even though the total always sums correctly.
- “Comparable” needs a consistent time-window definition (e.g., a store must have been open for at least 12 full months) — changing that criterion changes the numbers.
- Mix often ends up acting as a residual that absorbs measurement error from the other two effects; always sanity-check whether the mix figure is plausible given the business context.

10. A small worked example

A hypothetical chain had \$48 million in revenue the prior year and \$53.2 million in the current year — total growth of \$5.2 million (+10.8%). Of that, \$3.6 million came from stores opened during the period. Comparable growth was therefore \$1.6 million. Breaking that down: price contributed +\$2.1 million, volume contributed −\$1.4 million (fewer units sold at existing stores), and mix contributed +\$0.9 million. In other words, comparable growth only stayed positive because price and mix offset a real drop in volume — a warning sign the headline +10.8% figure alone did not reveal.



Illustrative data, not drawn from any real project.

Figure 20: Waterfall decomposition of revenue growth

11. Simple code example

```
import pandas as pd

# revenue by store, two periods, with units sold
df = pd.DataFrame({
    "store": ["A", "B", "C", "D"],
    "comparable": [True, True, True, False], # D is a new store
    "revenue_prior": [12.0, 15.0, 18.0, 0.0],
    "revenue_current": [12.8, 14.1, 19.7, 3.6],
    "units_prior": [2400, 3000, 3600, 0],
    "units_current": [2280, 2820, 3760, 720],
})

comp = df[df["comparable"]]

revenue_prior_total = df["revenue_prior"].sum()
revenue_current_total = df["revenue_current"].sum()
total_growth = revenue_current_total - revenue_prior_total

new_store_growth = df.loc[~df["comparable"], "revenue_current"].sum()
comparable_growth = (
    comp["revenue_current"].sum() - comp["revenue_prior"].sum()
)

price_prior = comp["revenue_prior"].sum() / comp["units_prior"].sum()
price_current = comp["revenue_current"].sum() / comp["units_current"].sum()

volume_effect = (comp["units_current"].sum() - comp["units_prior"].sum()) * price_prior
```

```
price_effect = (price_current - price_prior) * comp["units_current"].sum()
mix_effect = comparable_growth - volume_effect - price_effect

print(f"Total growth: {total_growth:.2f}")
print(f"  New stores: {new_store_growth:.2f}")
print(f"  Comparable: {comparable_growth:.2f}")
print(f"    Price: {price_effect:.2f} | Volume: {volume_effect:.2f} | Mix: {mix_effect:.2f}")
```

Poisson regression with fixed effects

1. What problem does this solve?

An analyst wants to predict how many goals a team scores in a match, as a function of the team's attacking strength, the opponent's defensive strength, and whether the match is home or away. The problem: goal counts are counts — 0, 1, 2, 3... — and an ordinary linear regression can predict nonsensical values like “−0.4 goals,” or treat a jump from 0 to 1 goal the same way as a jump from 5 to 6 goals, when in practice those jumps don't carry the same statistical weight.

2. Intuition

Counts of rare, discrete events — goals in a match, customers walking into a store per hour, machine failures per week — tend to follow a characteristic pattern: never negative, always whole numbers, and most values bunched near zero with a long right tail (seeing 0, 1, or 2 goals is far more common than seeing 6 or 7). A normal distribution (the symmetric bell curve) captures none of that — it allows negative values and is symmetric around the mean. The Poisson distribution was built precisely for this kind of data.

3. Plain explanation

Poisson regression models the expected number of events as an exponential function of the explanatory variables, guaranteeing the prediction is never negative. Instead of predicting the goal count directly, the model predicts the **logarithm** of the expected goal count as a linear combination of the variables — then converts back using the exponential. That choice (called the “log-linear form”) is what guarantees the final prediction is always positive, no matter what the coefficients turn out to be.

Fixed effects per team enter the model as an “individual adjustment” for each club — a way of asking “after accounting for how much this specific team typically scores or concedes, what's left of the effect I'm actually interested in (say, playing at home)?”

4. Simple conceptual example

Picture comparing the average number of customers walking into two stores per hour: one downtown (high foot traffic) and one in a residential neighborhood (low foot traffic). If you want to isolate the effect of “it's Friday” on customer flow, you first need to account for the fact that the downtown store already runs at a much higher baseline than the neighborhood one — otherwise any observed difference could simply reflect which store you're looking at, not the day of the week. The per-store fixed effect makes exactly that adjustment, store by store, before estimating the “Friday” effect.

5. How it works, broadly

1. For each observation (a match, an hour, a period), the expected number of events is written as $\lambda = e^{(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots)}$ — the weighted sum of explanatory variables, exponentiated.
2. A dummy (indicator variable) is created for each unit receiving a fixed effect (each team, for instance) — this lets every team have its own “baseline level” of expected goals, without forcing all teams to be identical in the absence of the other variables.

3. The model is fit by maximum likelihood, finding the coefficients that make the observed data most probable under the assumption that the count in each observation follows a Poisson distribution with mean λ .
4. Each estimated coefficient represents that variable's effect on the log scale of the expected event count.

6. What the result means

A variable's coefficient, once exponentiated (e^β), becomes a **multiplicative factor** on the expected number of events — not an additive one. If the coefficient for “playing at home” is 0.30, then $e^{0.30} \approx 1.35$: playing at home multiplies the expected goal count by 1.35, i.e., a 35% increase, holding everything else constant.

7. How to interpret it

Always translate the raw (log-scale) coefficient into the multiplicative factor before communicating the result — “the coefficient was 0.30” means nothing to a non-statistician, but “playing at home increases expected goals by 35%” is immediately interpretable. Multiplicative factors above 1 indicate an increase; below 1, a decrease (for example, 0.85 means a 15% drop).

8. When it's useful

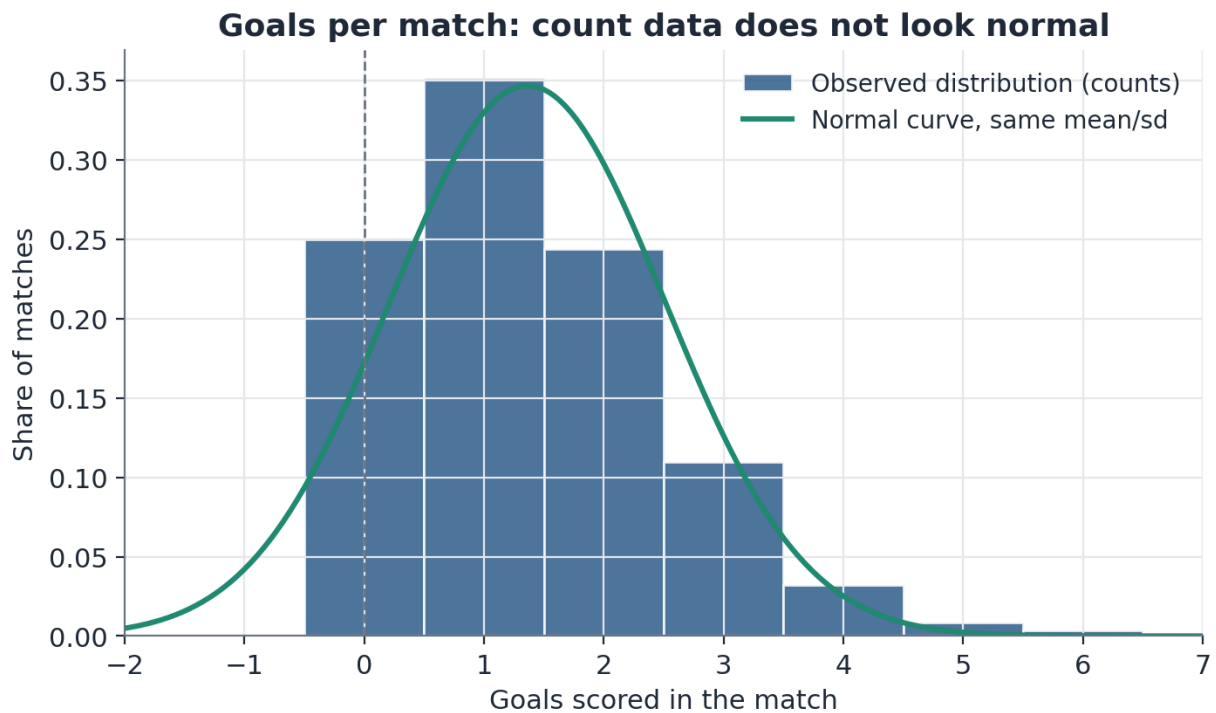
Whenever the response variable is an event count — not a proportion, not a continuous average. This is the type of model typically used in sports analytics to estimate how many goals a team is expected to score, combining the team's own attacking strength, the opponent's defensive strength, and the home-field effect, with per-club fixed effects isolating baseline differences between teams.

9. Important caveats

- Poisson assumes the variance of the count equals its mean (equidispersion). When the observed variance is clearly larger than the mean — **overdispersion** — the standard errors from a plain Poisson model come out artificially small, inflating apparent statistical significance. In those cases, alternative models (Poisson with robust standard errors, negative binomial) are more appropriate.
- Fixed effects per unit (team, store) consume degrees of freedom — with many units and few observations per unit, the model can become unstable.
- The model assumes independence between events conditional on the included variables; contexts with strong temporal dependence (a winning streak that shifts a team's confidence) violate that assumption.

10. A small worked example

Suppose a simplified model (without fixed effects, to illustrate the mechanics) where the expected goal count for a visiting team is $\lambda = e^{0.10} \approx 1.11$, and when playing at home, $\lambda = e^{0.10+0.30} = e^{0.40} \approx 1.49$. The ratio between the two, $1.49/1.11 \approx 1.35$, confirms that the home effect multiplies the expected goal count by 1.35 — a 35% increase — without needing to compute the plain difference between 1.49 and 1.11 (which would be just 0.38 goals, a less informative reading than the multiplicative factor).



Illustrative data, not drawn from any real project.

Figure 21: Goal-count distribution compared to a normal curve

11. Simple code example

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf

# one row per team-match: goals scored, home indicator, and teams involved
df = pd.DataFrame({
    "goals": [1, 2, 0, 3, 1, 2, 0, 1, 2, 1],
    "home": [1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
    "team": ["A", "A", "B", "B", "C", "C", "D", "D", "E", "E"],
})

model = smf.glm(
    formula="goals ~ home + C(team)",
    data=df,
    family=sm.families.Poisson(),
).fit()

print(model.summary())

# multiplicative factor for the home-field effect
```

```
home_coef = model.params["home"]  
print(f"Multiplicative factor (home field): {np.exp(home_coef):.2f}")
```

Weighted least squares (WLS) trend with confidence-interval classification

1. What problem does this solve?

An analyst wants to know, for every store in a chain, whether average ticket size is rising, falling, or holding steady over the last several quarters. The problem is that not every quarter has the same number of recorded sales — a quarter with 500 transactions is a far more reliable estimate than one with 20. An ordinary linear regression treats every point as equally trustworthy, letting a single sparse quarter distort the estimated trend just as much as a well-populated one.

2. Intuition

Imagine asking two people for a restaurant’s average rating: one answered based on 400 reviews, the other based on 3. Intuitively, you trust the first answer more. Weighted least squares (WLS) does exactly that when fitting a trend line: it gives more weight to points backed by more observations, and less weight to points backed by few.

3. Plain explanation

The method fits a line (or another simple trend) to the data over time, exactly like an ordinary linear regression — but instead of minimizing the plain sum of squared errors, it minimizes the **weighted** sum of squared errors, using each period’s number of observations as the weight. After estimating the line’s slope, a confidence interval is computed for that slope, and the trend is classified in a simple, objective way:

- If the slope’s confidence interval sits entirely **above zero** → an **upward** trend.
- If it sits entirely **below zero** → a **downward** trend.
- If the interval **includes zero** → a **stable** trend (there isn’t enough statistical evidence that the slope differs from zero).

4. Simple conceptual example

Two stores record their average ticket size over six quarters. Store A rises consistently, quarter after quarter, with plenty of sales in every period — the estimated slope is clearly positive and the confidence interval doesn’t come close to zero: classified as “rising.” Store B bounces up and down with no clear pattern, with few sales in some quarters — the estimated slope might even be slightly positive, but the confidence interval is wide enough to include zero: classified as “stable,” even though the fitted line isn’t perfectly flat.

5. How it works, broadly

1. For each unit (store, region, category), arrange the metric of interest over time (quarter, month) alongside the number of observations backing each point.
2. Fit a line using weighted least squares, where each point’s weight is proportional to its number of observations — points with more data pull the line more toward them.
3. Compute the standard error of the estimated slope, which already accounts for the weighting.
4. Build a confidence interval (typically 95%) for the slope.
5. Classify the trend as rising, falling, or stable, depending on where the confidence interval sits relative to zero.

6. What the result means

The estimated slope tells you how much, on average, the metric changes per period. The confidence interval tells you how precise that estimate is — wide intervals (common with few observations, high variability, or few time periods) make it harder to confidently claim a real trend exists, even if the point estimate of the slope looks different from zero.

7. How to interpret it

“Stable” in this classification doesn’t necessarily mean “no change whatsoever” — it means “there isn’t enough statistical evidence, given the observed volume and variability, to tell the slope apart from zero.” A store might genuinely be flat, or it might simply lack enough data to detect a real trend that exists but is small. It’s important not to conflate the two when communicating the result.

8. When it’s useful

Whenever you need to classify the trend of many units (stores, players, regions, products) systematically and comparably, especially when the number of observations per period varies substantially across units or over time. This is the logic used in sports analytics to classify whether a club’s performance is trending up, down, or holding steady over a season, weighting each period by the number of matches or events observed in that stretch.

9. Important caveats

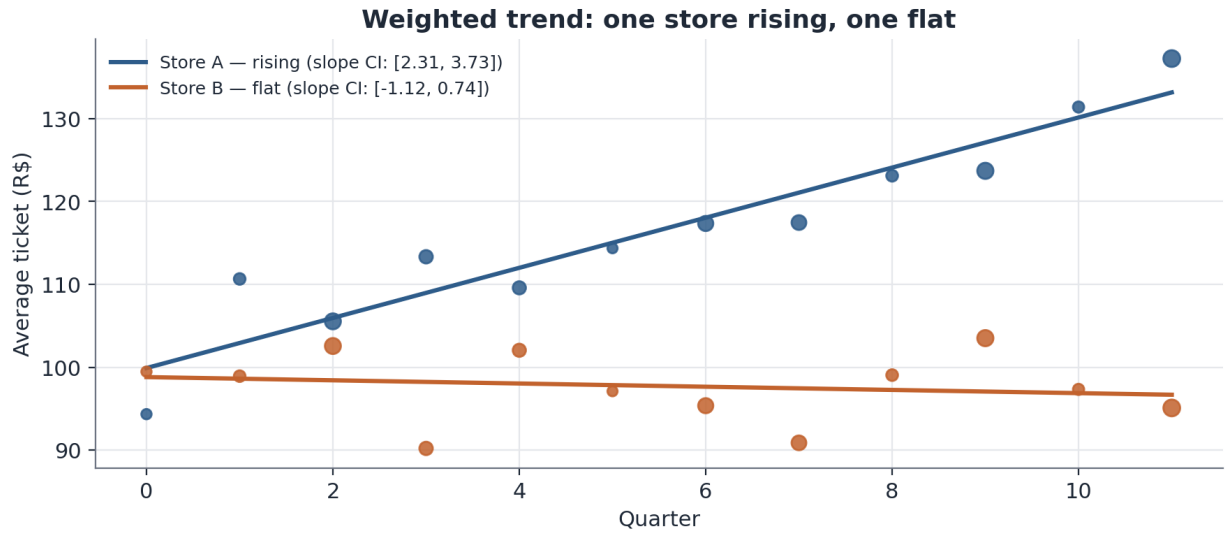
- CI-based classification depends on the chosen confidence level (95% is common but not universal) — a 90% CI classifies more series as “rising” or “falling” than a 99% CI would, for the same data.
- Few time points (say, only three or four periods) make the confidence interval very wide, making any classification other than “stable” hard to reach even when a real trend exists.
- WLS assumes a linear relationship with time; non-linear trends (growth that accelerates or decelerates) can be poorly captured by a single line.
- Weighting by number of observations corrects for unequal sampling precision, but it doesn’t fix data-quality issues within a given period (for example, a period with many transactions but a systematic recording error).

10. A small worked example

Two stores, twelve quarters each, with observation counts ranging from 40 to 400 per quarter. For Store A, the WLS-estimated slope is +2.4 (metric units per quarter), with a 95% confidence interval between +1.1 and +3.7 — entirely above zero, classified as **rising**. For Store B, the estimated slope is +0.05, with a confidence interval between −1.2 and +1.3 — it crosses zero, classified as **stable**, even though the point estimate is technically positive.

11. Simple code example

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
```



Illustrative data, not drawn from any real project.

Figure 22: Weighted trend classification for two stores

```
df = pd.DataFrame({
    "quarter": range(12),
    "avg_ticket": [100, 102, 105, 104, 108, 110, 111, 115, 116, 119, 121, 124],
    "n_observations": [40, 55, 380, 390, 60, 400, 45, 370, 50, 360, 400, 390],
})

X = sm.add_constant(df["quarter"])
model = sm.WLS(df["avg_ticket"], X, weights=df["n_observations"]).fit()

slope = model.params["quarter"]
ci_low, ci_high = model.conf_int(alpha=0.05).loc["quarter"]

if ci_low > 0:
    classification = "rising"
elif ci_high < 0:
    classification = "falling"
else:
    classification = "stable"

print(f"Slope: {slope:.2f} | 95% CI: [{ci_low:.2f}, {ci_high:.2f}] | Classification: {classification}")
```

Logistic regression

1. What problem does this solve?

An analyst wants to predict whether something happens or not — a customer cancels or not, a team gets relegated or not, a patient responds to treatment or not — from one or more continuous variables. The problem: the outcome is binary (0 or 1), and an ordinary linear regression fit directly to that 0/1 variable can predict values like -0.3 or 1.4 , which don't correspond to any real probability.

2. Intuition

A probability needs to stay between 0 and 1, always. Logistic regression solves this by modeling, not the probability directly, but a transformation of it — the **log odds** — as a linear combination of the explanatory variables. That linear combination can take any value, from minus to plus infinity; the transformation back (the logistic function, S-shaped) compresses it into the $(0, 1)$ interval, guaranteeing the model's output is always a valid probability.

3. Plain explanation

For each combination of explanatory variables, the model estimates the probability that the outcome equals 1. The curve relating the linear combination of variables to the probability is S-shaped (sigmoid): near the middle, small changes in the variable shift the predicted probability a lot; at the extremes (near 0 or near 1), the same change has almost no effect — the probability is already near the boundary and can't cross it.

4. Simple conceptual example

Think of predicting whether a student passes an exam from how many hours they studied. Someone who studied 0 hours almost certainly fails; someone who studied 20 hours almost certainly passes — at both extremes, one more or fewer hour barely changes the prediction. It's in the middle range (say, between 4 and 8 hours) that an extra hour of study makes the biggest difference to the probability of passing — exactly the region where the S-curve is steepest.

5. How it works, broadly

1. For each observation, the model computes a linear combination of the explanatory variables:
$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$
2. That combination passes through the logistic function, $p = 1/(1 + e^{-z})$, which converts any real number into a probability between 0 and 1.
3. The β coefficients are estimated by maximum likelihood — the set of values that makes the observed outcomes (who had 1, who had 0) most probable under the model.
4. To classify a new observation, the predicted probability is compared to a threshold (0.5 by default) — above it, 1 is predicted; below it, 0. The threshold can be adjusted depending on the cost of each type of error.

6. What the result means

Each coefficient, exponentiated (e^β), becomes an **odds ratio**: how much the odds of the outcome being 1 gets multiplied by when the variable increases by one unit, holding everything else constant.

An odds ratio of 2.0 means the odds double; one of 0.5 means they get cut in half. “Odds” here is the ratio $p/(1-p)$, not the probability p itself — the distinction matters because odds ratios and probabilities don’t move in the same proportion.

7. How to interpret it

Always translate the raw coefficient (on the log-odds scale) into an odds ratio before communicating it — “the coefficient was 0.69” says nothing on its own, but “each extra point of satisfaction cuts the odds of cancelling in half” ($e^{-0.69} \approx 0.50$) is immediately interpretable. To assess the model’s quality as a classifier — not just the significance of its coefficients — two common metrics are **AUC** (area under the ROC curve: the probability the model scores a random positive observation higher than a random negative one; 0.5 is random, 1.0 is perfect separation) and the **Brier score** (mean squared error between predicted probability and observed outcome; lower is better calibrated).

8. When is it useful

Whenever the outcome of interest is binary and you want both to **quantify the effect** of each explanatory variable (via the odds ratio) and to **predict a probability** for new cases — not just a 0/1 classification. It’s the model of choice for turning a continuous indicator (points pace, satisfaction score, usage time) into an actual chance that a binary outcome occurs.

9. Important caveats

- **Quasi-separation:** when a variable almost perfectly separates the two classes (say, everyone above some value has outcome 1 and everyone below has 0), the maximum-likelihood algorithm doesn’t converge stably — coefficients and their standard errors become artificially large. This tends to happen precisely when the model is working well, and it doesn’t invalidate the direction of the effect, but it calls for caution when interpreting the confidence interval.
- **Correlation between observations:** if several observations come from the same unit (the same customer across different months, the same team across different rounds of a season), treating them as independent artificially inflates the model’s confidence. In those cases, validating with a split that respects the group structure (e.g., holding out one whole unit at a time) is more reliable than trusting the in-sample fit alone.
- **Significance isn’t accuracy:** a coefficient can be statistically significant (small p-value) while the model still has little power to separate the classes (AUC close to 0.5) — these are different questions.

10. A small worked example

Suppose a model with a single variable (x = satisfaction score from 0 to 10) predicting cancellation, with coefficient $\hat{\beta} = -0.69$ (standard error 0.10) and intercept $\hat{\beta}_0 = 3.45$. The odds ratio is $e^{-0.69} \approx 0.50$: each extra point of satisfaction cuts the odds of cancelling in half. The point where predicted probability equals 50% sits at $x = -\hat{\beta}_0/\hat{\beta} = 5.0$ — below that, the model predicts a higher chance of cancelling than of staying; above it, the reverse.

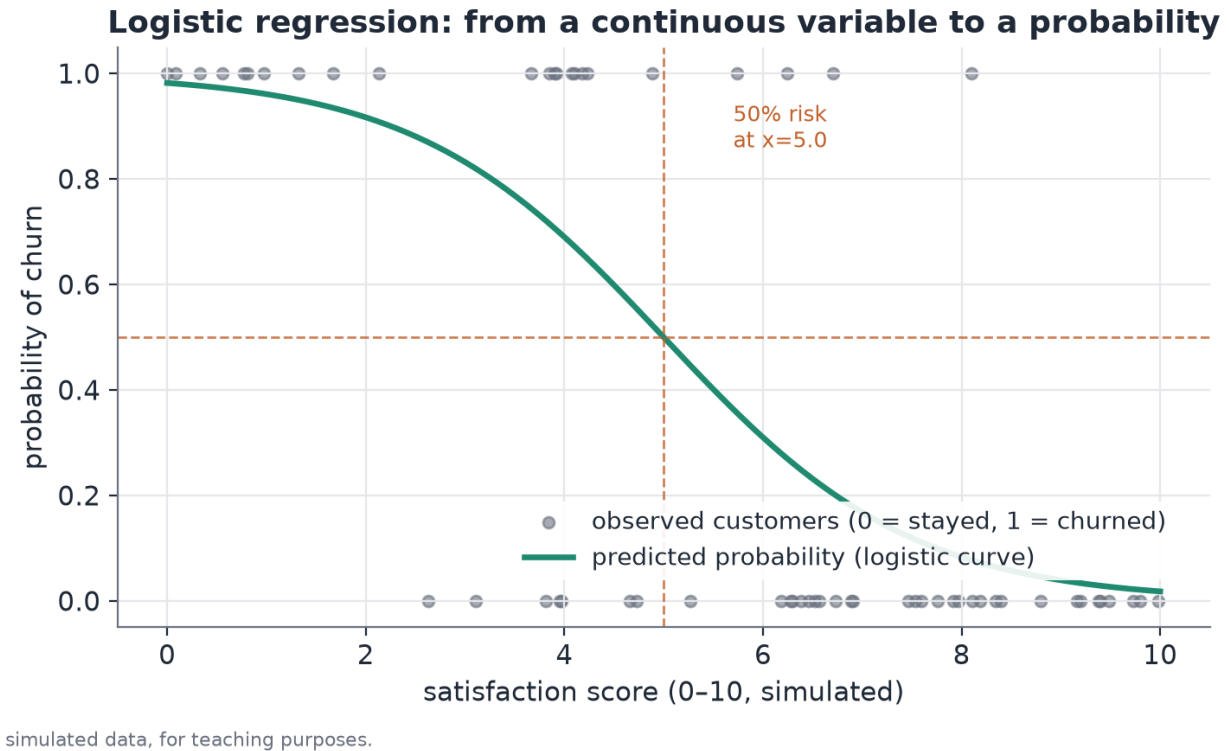


Figure 23: Logistic regression: from a continuous variable to a probability

11. Simple code example

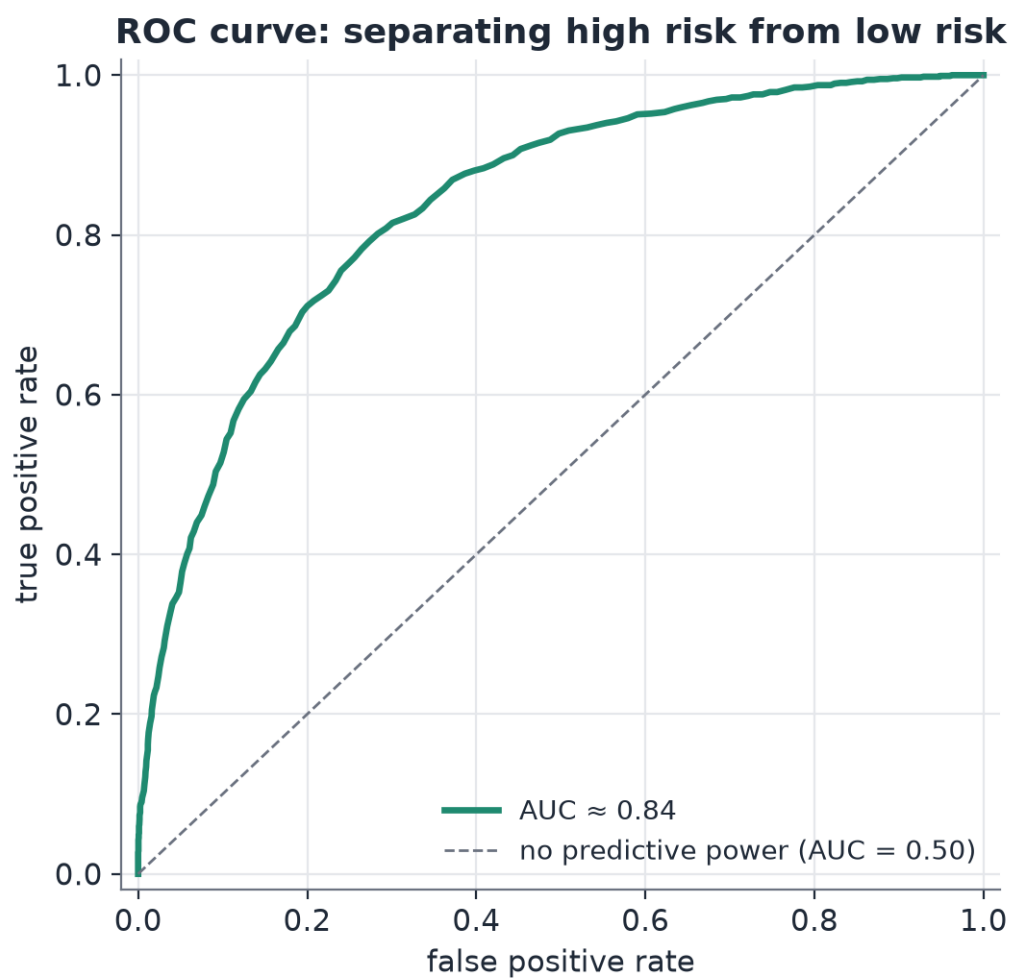
```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# one row per customer: satisfaction score and whether they cancelled (1) or not (0)
df = pd.DataFrame({
    "satisfaction": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "cancelled":     [1, 1, 1, 1, 0, 1, 0, 0, 0, 0],
})

X = sm.add_constant(df["satisfaction"])
model = sm.Logit(df["cancelled"], X).fit()
print(model.summary())

# odds ratio for each extra point of satisfaction
print(f"Odds ratio: {np.exp(model.params['satisfaction']):.2f}")

# predicted probability for a customer with satisfaction = 6
print(model.predict([1, 6]))
```



illustrative classifier, simulated data.

Figure 24: ROC curve for an illustrative classifier